

OBJECT ORIENTED PROGRAMMING **PROJECT**

→ **Project Genre** :-

System Building

→ **Project Name** :-

Smart City Automation

Project Introduction and Details

Introduction :-

Smart City Automation is a system that depicts a whole life in a city how Humans live . What are the essentials resources for a Human life that need to be present in a city . How the life goes in a city this system refers to that . In this we have many modules that are most important parts of city . And also how a city works in

reality . This project is basically mimics the reality of a smart city into programming using the concepts of OOP .

PROJECT DETAILS

→ Module 1 :- Building System

→ Classes , Structures , Relationships , Inheritance and Polymorphism

1.1 Building (Abstract Base Class)

Purpose :- Represents any structure in the city. This is the foundation of the building hierarchy using **abstraction** and **polymorphism**.

Attributes :-

protected:

string buildingID; → Unique identifier (e.g "BLD-001")
string address; → Street address (e.g "123 Main St")
int buildYear; → Year constructed (e.g., 2020)
double landArea; → Area in square meters
double buildingValue; → Current market value in dollars
int condition; → 1-100 scale (100=perfect condition)
bool isOperational; → Whether building is functional
int floors; → Number of floors
double maintenanceCost; → Annual maintenance cost

private:

static int buildingCounter; → Static counter for unique IDs

Methods :-

public:

.) Constructor

Building(string address , int year, double area, int floors);

.) Pure virtual methods (MUST be implemented by derived classes)

```
virtual double calculateMaintenanceCost() = 0;  
virtual int getDailyVisitors() = 0;  
virtual string getBuildingType() const = 0;  
virtual void displayDetails() const = 0;
```

.) Virtual methods (CAN be overridden)

```
virtual void upgrade();  
virtual void deteriorate();  
virtual double calculatePropertyTax() const;
```

.) Regular methods (shared by all buildings)

```
int getAge() const;  
bool needsMaintenance() const;  
void performMaintenance();
```

.) Getters/Setters (Encapsulation)

```
string getAddress() const;  
double getValue() const;  
void setValue(double value);  
int getCondition() const;
```

.) Virtual destructor (for proper cleanup in inheritance)

```
virtual ~Building();
```

OOP Concepts Used:

1. Abstraction:

- Building is an **abstract class** (has pure virtual functions)
- Cannot create Building objects directly
- Forces derived classes to implement specific behaviors
- Hides implementation details, shows only essential features

2. Encapsulation:

- Private data members (buildingCounter)
- Protected data members (accessible to derived classes)
- Public interface through getters/setters
- Validation in setters to maintain data integrity

3. Polymorphism:

- Pure virtual functions allow different building types to behave differently
- Base class pointer can point to any derived class object
- Runtime polymorphism through virtual functions

4. Static Members:

- static int buildingCounter - shared across all Building instances
- Used to generate unique building IDs

1.2) ResidentialBuilding (Derived from Building)

Purpose:

Intermediate class for all residential structures. Adds common residential features.

Inheritance:

Building (Base)



ResidentialBuilding (Derived)

Attributes:-

protected:

int maxOccupants; → Maximum people who can live here

int currentOccupants; → Current residents

vector<Citizen*> residents; → People living here (Composition)

bool hasParking; → Parking availability

bool hasGarden; → Garden/yard

double monthlyRent; → For rental properties

bool isRented; → Rental status

private:

double utilityBill; → Monthly Utility Costs

Methods :-

public:

ResidentialBuilding(string address, int year, double area, int floors, int maxOcc);

virtual double calculateMaintenanceCost() override;

virtual string getBuildingType() const override;

→ New virtual methods for residential

virtual double calculateRent() const;

virtual bool canAccommodate(int people) const;

→ Resident management

void addResident(Citizen* citizen);

void removeResident(Citizen* citizen);

vector<Citizen*> getResidents() const;

→ Utility

bool hasVacancy() const;

int getVacantSpots() const;

→ Destructor

virtual ~ResidentialBuilding();

OOP Concepts Used:

1. Inheritance:

- Inherits from Building
- Uses override keyword to implement base class pure virtuals
- Adds residential-specific functionality

2. Composition:

- HAS-A relationship with Citizen objects
- `vector<Citizen*>` - contains citizens but doesn't own their lifecycle
- Demonstrates strong relationship (residential building contains residents)

3. Method Overriding:

- Provides specific implementation of `calculateMaintenanceCost()`
- Calls base class method and adds residential-specific costs

1.3 House (Derived from ResidentialBuilding)

Purpose:

Represents a single-family home. Most specific residential type.

Inheritance Chain:

Building (Grandparent)



ResidentialBuilding (Parent)



House (Child)

Attributes :-

private:

int numBedrooms;	Number of bedrooms
int numBathrooms;	Number of bathrooms
bool hasGarage;	Garage presence
int garageCapacity;	Car spaces in garage
bool hasBasement;	Basement presence
bool hasAttic;	Attic presence
string architectureStyle;	Victorian, Modern, Colonial, etc.
double gardenArea;	Garden size in sq meters

Methods :-

public:

House(string address, int year, double area, int bedrooms, int bathrooms);

```
double calculateMaintenanceCost() override;
int getDailyVisitors() override;
void displayDetails() const override;
double calculateRent() const override;

void addGarage(int capacity);
void renovateKitchen();
void addBasement();

int getNumBedrooms() const;
int getNumBathrooms() const;
bool hasGarageSpace() const;

~House();
```

OOP Concepts Used:

1. Multi-Level Inheritance:

- Inherits from ResidentialBuilding, which inherits from Building
- Has access to all public/protected members of both parents
- Demonstrates inheritance chain

2. Method Overriding:

- Overrides methods from both ResidentialBuilding and Building
- Uses override keyword for compile-time checking

3. Calling Parent Methods:

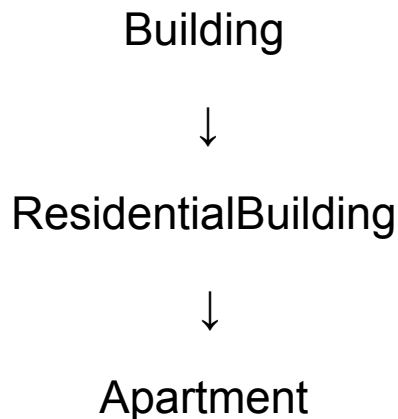
- Uses ResidentialBuilding::calculateMaintenanceCost() to reuse parent logic
- Demonstrates code reuse through inheritance

1.4 Apartment (Derived from ResidentialBuilding)

Purpose:

Multi-unit residential building with multiple families.

Inheritance:



Attributes :-

private:

int totalUnits;	Total apartment units
int occupiedUnits;	Currently occupied
vector<int> unitSizes;	Size of each unit (sq meters)
vector<double> unitRents;	Rent for each unit
bool hasElevator;	Elevator presence
bool hasSecurity;	Security guard
int parkingSpots;	Total parking spaces
int availableParking;	Available parking
bool hasGym;	Gym facility
bool hasPool;	Swimming pool

Methods :-

public:

Apartment(string address, int year, double area, int floors,
int units);

Override methods

double calculateMaintenanceCost() override;

int getDailyVisitors() override;

void displayDetails() const override;

double calculateRent() const override;

Apartment-specific

bool rentUnit(int unitNumber, Citizen* tenant);

void vacateUnit(int unitNumber);

int getAvailableUnits() const;

double getOccupancyRate() const;

void addAmenity(string amenity); // Add gym, pool, etc.

~Apartment();

OOP Concepts Used :-

1. Sibling Inheritance:

- House and Apartment are siblings (both inherit from ResidentialBuilding)
- Share common parent but have different implementations
- Demonstrates polymorphism - same parent, different behaviors

2. Container Usage:

- `vector<int>` and `vector<double>` for managing multiple units
- Shows practical use of STL containers

1.5 TowerBlock (Derived from ResidentialBuilding)

Purpose:

High-rise residential building with many floors.

Inheritance:

Building



ResidentialBuilding



TowerBlock

Attributes :-

private:

```
int highestFloor;      Number of floors
int unitsPerFloor;     Apartments per floor
vector<vector<Citizen*>> floorResidents; 2D vector of
residents
int elevatorCount;     Number of elevators
bool hasPenthouse;     Top floor luxury unit
double penthousePrice; Premium price
bool hasHelipad;       For extreme luxury
int securityLevel;     1-5 scale
```

```
struct FloorInfo { Use Struct to combine all Floor Attributes
    int floorNumber;
    int units;
    double avgRent;
    bool fullyOccupied;
};
vector<FloorInfo> floors;
```

Methods :-

public:

TowerBlock(string address, int year, double area, int floorCount, int unitsPerFloor);

double calculateMaintenanceCost() override;

int getDailyVisitors() override;

void displayDetails() const override;

void assignToFloor(int floor, Citizen* resident);

vector<Citizen*> getFloorResidents(int floor) const;

bool isFloorFull(int floor) const;

double getFloorOccupancyRate(int floor) const;

~TowerBlock();

OOP Concept Used :-

1. Nested Data Structures:

- vector<vector<Citizen*>> - 2D vector for floor organization
- Demonstrates complex data organization

2. Struct within Class:

- FloorInfo struct for organizing floor data
- Shows encapsulation of related data

1.6 CommercialBuilding (Derived from Building)

Purpose:

Base class for all commercial structures.

Inheritance:

Building



CommercialBuilding

Attributes :-

protected:

string businessType; Retail, Office, Restaurant, etc.

int employeeCapacity; Maximum workers

vector<Citizen*> employees; Current employees

double monthlyRevenue; Average monthly income

double operatingCost; Monthly expenses

bool isOpen; Currently operating
string openingHours; e.g., "9AM-6PM"
int customerCapacity; Max customers at once
bool hasParking; Customer parking
int parkingSpaces; Number of spots

private:

double monthlyProfit; Revenue - costs
vector<string> services; Services offered

Methods :-

public:

CommercialBuilding(string addr, int year, double area,
string bizType);

virtual double calculateMaintenanceCost() override;
virtual string getBuildingType() const override;

Commercial-specific virtuals

virtual double calculateMonthlyRevenue() = 0;
virtual double calculateBusinessTax() const;
virtual void hirEmployee(Citizen* employee);
virtual void fireEmployee(Citizen* employee);

Business operations
void open();
void close();
bool isOperatingNow() const;
double getProfit() const;

virtual ~CommercialBuilding();

OOP Concept Used :-

OOP Concepts Used:

1. Intermediate Abstract Class:

- Not fully concrete (has pure virtual calculateMonthlyRevenue)
- Not fully abstract (has some implemented methods)
- Demonstrates partial implementation

2. Association:

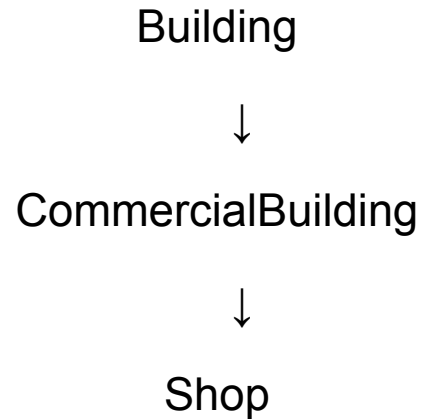
- HAS-A employees (Citizen*)
- Employees exist independently of building
- Weaker relationship than composition

1.7 Shop (Derived from CommercialBuilding)

Purpose:

Retail store selling goods.

Inheritance:



Attributes :-

private:

```
    string shopName;           // Business name
    string shopCategory;       // Groceries, Electronics,
    Clothing, etc.

    struct Product {
```

```
    string name;  
    double price;  
    int quantity;  
    string category;  
};  
vector<Product> inventory;  Products for sale
```

```
int dailyCustomers;    Average daily footfall  
double avgTransactionValue;  Average sale amount  
bool hasOnlineStore;    E-commerce presence  
double onlineRevenue;    Online sales
```

Methods :-

```
public:
```

Shop(string addr, int year, double area, string name, string category);

double calculateMaintenanceCost() override;

int getDailyVisitors() override;

void displayDetails() const override;

double calculateMonthlyRevenue() override;

void addProduct(string name, double price, int quantity);

void sellProduct(string name, int quantity);

void restockProduct(string name, int quantity);

int getInventoryCount() const;

double getAverageCustomerValue() const;

vector<Product> getBestSellers() const;

~Shop();

OOP Concepts Used :-

1. Struct within Class:

- Product struct encapsulates product data
- Demonstrates data organization

2. Vector of Structs:

- `vector<Product>` manages inventory
- Shows practical data structure usage

1.8 Restaurant (Derived from CommercialBuilding)

Purpose:

Food service establishment.

Inheritance:

Building



CommercialBuilding



Restaurant

Attributes :-

private:

string restaurantName;

string cuisineType; Italian, Chinese, American, etc.

int seatingCapacity; Number of seats

int currentDiners; Current customers

struct MenuItem {

 string name;

 double price;

 string category; Appetizer, Main, Dessert

 int preparationTime; Minutes

};

vector<MenuItem> menu;

double avgMealPrice; Average bill per customer

int dailyCustomers; Daily foot traffic

double foodCost; Cost of ingredients

int chefCount; Number of chefs

int waiterCount; Number of waiters

Ratings

double foodQuality; 1-5 stars

double serviceQuality; 1-5 stars

double ambiance; 1-5 stars

Methods :-

public:

Restaurant(string addr, int year, double area, string name, string cuisine, int seats);

double calculateMaintenanceCost() override;

int getDailyVisitors() override;

void displayDetails() const override;

double calculateMonthlyRevenue() override;

Restaurant operations

void addMenuItem(string name, double price, string category);

void removeMenuItem(string name);

bool seatCustomers(int count);

```
void serveCustomers(int count);
```

Management

```
void hireChef();
```

```
void hireWaiter();
```

```
void updateRating(double food, double service, double  
amb);
```

```
double get Overall Rating() const;
```

```
~Restaurant();
```

OOP Concepts Used :

1. Specialized Behavior:

- Implements business logic specific to restaurants
- Shows how inheritance allows specialization

2. Multiple Data Members:

- Combines inventory (menu), employees, customers
- Demonstrates complex object modeling

1.9 Office (Derived from CommercialBuilding)

Purpose:

Professional workspace building.

Inheritance :

Building



CommercialBuilding



Office

Attributes : -

private:

```
string companyName;  
string industry;           Tech, Finance, Legal, etc.  
int totalFloors;  
int cubiclesPerFloor;  
int privateOffices;  
int conferenceRooms;
```

```
struct OfficeSpace {  
    int floor;  
    int number;  
    bool isOccupied;
```

```
Citizen* occupant;  
string department;  
};  
vector<OfficeSpace> spaces;  
  
bool hasReception;  
bool hasCafeteria;  
bool hasGym;  
int meetingRoomsCount;  
double rentPerSqMeter;    Office space rental rate
```

Methods :-

```
public:  
    Office(string addr, int year, double area, string company,  
string industry);
```

```
double calculateMaintenanceCost() override;  
int getDailyVisitors() override;  
void displayDetails() const override;  
double calculateMonthlyRevenue() override;
```

Office management

```
void assignOffice(int floor, int number, Citizen* employee);  
void vacateOffice(int floor, int number);
```

```
bool hasAvailableSpace() const;  
int getOccupancyCount() const;
```

Amenities

```
void addConferenceRoom();  
void addCafeteria();
```

```
~Office();
```

1.10 Mall (Derived from CommercialBuilding)

Purpose:

Shopping complex with multiple stores.

Inheritance:

Building



CommercialBuilding



Mall

Attributes :-

private:

```
string mallName;
int totalStores;
int occupiedStores;

struct StoreSpace {
    int storeNumber;
    double area;
    bool isRented;
    Shop* tenant;           Composition with Shop
    double monthlyRent;
};
vector<StoreSpace> stores;

int foodCourtCapacity;
int restaurantCount;
bool hasCinema;
int cinemaScreens;
int parkingLevels;
int totalParkingSpots;

double footTraffic;        Daily visitors
bool hasSecuritySystem;
```

Methods :-

public:

```
Mall(string addr, int year, double area, string name, int storeCount);
```

```
double calculateMaintenanceCost() override;  
int getDailyVisitors() override;  
void displayDetails() const override;  
double calculateMonthlyRevenue() override;
```

Mall operations

```
void rentStoreSpace(int storeNum, Shop* shop);  
void vacateStore(int storeNum);  
int getVacancyCount() const;  
double getOccupancyRate() const;
```

Amenities

```
void addCinema(int screens);  
void addFoodCourt(int capacity);
```

```
~Mall();
```

OOP Concepts Used :-

1. Composition:

- Mall contains Shop objects

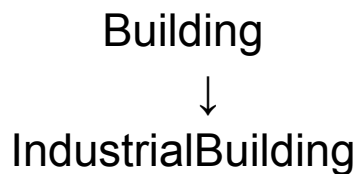
- Strong "owns" relationship
- Demonstrates complex object relationships

1.11 IndustrialBuilding (Derived from Building)

Purpose:

Base for manufacturing and production facilities.

Inheritance:



Attributes :-

protected:

string industryType; Manufacturing, Processing,
Assembly

int workerCapacity; Max workers

vector<Citizen*> workers; Current workforce

double productionCapacity; Units per day

double currentProduction; Actual production

double pollutionLevel; Environmental impact (1-10)
bool hasWasteManagement; Waste disposal system
double energyConsumption; kWh per day
double waterConsumption; Liters per day

private:

double rawMaterialCost; Material expenses
double productionCost; Operating costs
double productValue; Revenue from products

Methods :-

public:

IndustrialBuilding(string addr, int year, double area, string industry);

virtual double calculateMaintenanceCost() override;
virtual string getBuildingType() const override;

Production virtuals

virtual void produce() = 0;
virtual double calculateProductionCost() = 0;

Worker management

```
void hireWorker(Citizen* worker);  
void layoffWorker(Citizen* worker);
```

Environmental

```
double getPollutionLevel() const;  
void installWasteManagement();
```

```
virtual ~IndustrialBuilding();
```

OOP Concepts Used :-

1. Environmental Modeling:

- Demonstrates real-world constraints in OOP
- Pollution, energy, water consumption

1.12 Factory (Derived from IndustrialBuilding)

Purpose:

Manufacturing facility producing goods.

Inheritance:

Building



IndustrialBuilding



Factory

Attributes :-

private:

string productType; // What factory produces

```
struct ProductionLine {  
    string lineID;  
    string productName;  
    int unitsPerHour;  
    int workersRequired;  
    bool isOperational;  
    double maintenanceCost;
```

```
};
```

```
vector<ProductionLine> productionLines;
```

int shiftCount; 1, 2, or 3 shifts

int hoursPerShift; Usually 8 hours

```
struct RawMaterial {  
    string materialName;
```

```
double quantity;  
double pricePerUnit;  
double reorderLevel;    When to restock  
};  
vector<RawMaterial> materials;  
  
int dailyOutput;        Units produced per day  
double qualityRating;   Product quality (1-100)
```

Methods :-

```
public:  
    Factory(string addr, int year, double area, string  
productType);  
  
double calculateMaintenanceCost() override;  
int getDailyVisitors() override;  
void displayDetails() const override;  
  
void produce() override;  
double calculateProductionCost() override;
```

Factory operations

```
void addProductionLine(string lineID, string product, int  
unitsPerHour);  
void shutdownLine(string lineID);  
void startLine(string lineID);
```

Materials

```
void orderRawMaterial(string material, double quantity);  
bool hasEnoughMaterials() const;
```

Efficiency

```
double getProductionEfficiency() const;  
void improveQuality();
```

```
~Factory();
```

OOP Concepts Used :-

Complex Data Modeling:

- Multiple structs for different aspects
- Shows real-world complexity in objects

1.13 Warehouse (Derived from IndustrialBuilding)

Purpose:

Storage and distribution facility.

Inheritance:

Building



IndustrialBuilding



Warehouse

Attributes :-

private:

double storageCapacity;	Total cubic meters
double usedStorage;	Currently used space

```
struct StoredItem {  
    string itemID;  
    string itemName;  
    double quantity;  
    string category;  
    double valuePerUnit;  
    string owner;      Company/business name  
};  
vector<StoredItem> inventory;
```

int loadingDocks; Number of loading bays
bool hasRefrigeration; Cold storage
bool hasClimateControl; Temperature/humidity control
int forkliftCount; Material handling equipment

Methods :-

public:

Warehouse(string addr, int year, double area, double capacity);

double calculateMaintenanceCost() override;

int getDailyVisitors() override;

void displayDetails() const override;

void produce() override; In warehouse, this means process shipments

double calculateProductionCost() override;

Storage operations

void storeItem(string itemID, string name, double quantity, string owner);

void retrieveItem(string itemID, double quantity);

```
double getAvailableSpace() const;  
double getUtilizationRate() const;
```

```
~Warehouse();
```

1.14 PublicServiceBuilding (Derived from Building)

Purpose:

Base for government and public service buildings.

Inheritance:

Building



PublicServiceBuilding

Attributes :-

protected:

string serviceName; Fire, Police, Education, Health

int staffCapacity; Maximum employees

vector<Citizen*> staff; Current staff

double annualBudget; Government funding

double operatingExpenses; Yearly costs

bool isFunded; Budget approval status

int serviceRadius; Area covered (blocks)
double serviceQuality; 1-100 rating

Methods :-

public:

 PublicServiceBuilding(string addr, int year, double area,
string service);

virtual double calculateMaintenanceCost() override;
virtual string getBuildingType() const override;

Service virtuals

virtual void provideService() = 0;
virtual int getServiceCapacity() const = 0;

Staff management

void hireStaff(Citizen* employee);
void dismissStaff(Citizen* employee);

Budget

void allocateBudget(double amount);
bool hasSufficientFunding() const;

```
virtual ~PublicServiceBuilding();
```

OOP Concepts Used :-

1. Public Service Modeling:

- Government-owned buildings
- Budget-dependent operations

1.15 School (Derived from PublicServiceBuilding)

Purpose:

Educational institution.

Inheritance:

Building



PublicServiceBuilding



School

Attributes :-

private:

string schoolName;

string schoolLevel; Elementary, Middle, High,

University

int studentCapacity; Maximum students

int currentStudents; Enrolled students

vector<Student*> students; Student list

int teacherCount; Number of teachers

int classroomCount; Number of classrooms

double studentTeacherRatio; Quality metric

struct Department {

string name; Math, Science, Arts

int teachers;

int students;

double budget;

};

vector<Department> departments;

bool hasLibrary;

```
bool hasLaboratory;  
bool hasSportsField;  
bool hasCanteen;
```

```
double academicRating;    School performance (1-100)  
double graduationRate;    Percentage
```

Methods :-

```
public:  
    School(string addr, int year, double area, string name,  
string level);
```

```
double calculateMaintenanceCost() override;  
int getDailyVisitors() override;  
void displayDetails() const override;
```

```
void provideService() override;  
int getServiceCapacity() const override;
```

Student management

```
void enrollStudent(Student* student);  
void graduateStudent(Student* student);  
int getEnrollmentCount() const;
```

Academic

```
void hireTeacher();  
void updateRating(double rating);  
double getStudentTeacherRatio() const;  
  
~School();
```

1.16 Hospital (Derived from PublicServiceBuilding)

Purpose:

Healthcare facility.

Inheritance:

Building



PublicServiceBuilding



Hospital

Attributes :-

private:

string hospitalName;

int totalBeds; Bed capacity

int occupiedBeds; Current patients

```
int emergencyBeds;    ER beds
int ICUBeds;          Critical care
```

```
struct Department {
    string name;      Cardiology, Orthopedics, etc.
    int doctors;
    int nurses;
    int beds;
    double budget;
};
vector<Department> departments;
```

```
int doctorCount;
int nurseCount;
int ambulanceCount;
```

```
bool hasEmergencyRoom;
bool hasSurgery;
bool hasICU;
bool hasRadiology;
```

```
double patientSatisfaction; 1-100
int dailyPatients;           Average daily admissions
```

Methods :-

public:

Hospital(string addr, int year, double area, string name, int
beds);

double calculateMaintenanceCost() override;
int getDailyVisitors() override;
void displayDetails() const override;

void provideService() override;
int getServiceCapacity() const override;

Patient care

bool admitPatient();
void dischargePatient();
int getAvailableBeds() const;
double getOccupancyRate() const;

Staff

void hireDoctor();
void hireNurse();

Emergency

bool canHandleEmergency() const;

~Hospital();

1.17 FireStation (Derived from PublicServiceBuilding)

Purpose:

Fire and rescue services.

Inheritance:

Building



PublicServiceBuilding



FireStation

Attributes :-

private:

string stationName;

int firefighterCount; Total firefighters

int onDutyFirefighters; Current shift

struct FireTruck {

 string truckID;

 bool isAvailable;

 int waterCapacity; Liters

 int crewSize; Firefighters on truck

 double maintenanceCost;

};


```
vector<FireTruck> trucks;
```

```
int totalTrucks;
```

```
int availableTrucks;
```

```
double responseTime;    Average minutes to reach fire
```

```
int annualFireCalls;    Incidents per year
```

```
double successRate;    Fires contained successfully
```

Methods :-

```
public:
```

```
    FireStation(string addr, int year, double area, string name);
```

```
    double calculateMaintenanceCost() override;
```

```
    int getDailyVisitors() override;
```

```
    void displayDetails() const override;
```

```
    void provideService() override;
```

```
    int getServiceCapacity() const override;
```

Operations

```
void dispatchTruck(string incidentLocation);
```

```
void recallTruck(string truckID);
```

```
bool hasTruckAvailable() const;
```

Management

```
void addFireTruck();
```

```
void hireFirefighter();
```

```
~FireStation();
```

1.18 PoliceStation (Derived from PublicServiceBuilding)

Purpose:

Law enforcement facility.

Inheritance:

Building



PublicServiceBuilding



PoliceStation

Attributes :-

private:

```
    string stationName;
```

```
    int officerCount;        Total police officers
```

```
    int onDutyOfficers;      Current shift
```

```
    struct PatrolCar {
```

```
        string carID;
```

```
        bool isAvailable;
```

```
    int officersAssigned;  
    string patrolZone;  
};  
vector<PatrolCar> patrolCars;
```

```
int cellCount;           Holding cells  
int detaineesCount;      Current detainees  
double crimeRate;        Area crime statistics  
double responseTime;     Average response minutes  
int annualCrimeCalls;    Incidents per year
```

Methods :-

```
public:  
    PoliceStation(string addr, int year, double area, string  
name);
```

```
double calculateMaintenanceCost() override;  
int getDailyVisitors() override;  
void displayDetails() const override;
```

```
void provideService() override;  
int getServiceCapacity() const override;
```

Operations

```
void dispatchOfficer(string incidentLocation);
```

```
void recallOfficer(string carID);  
bool hasOfficerAvailable() const;
```

Detention

```
void detainSuspect();  
void releaseSuspect();
```

```
~PoliceStation();
```

Building Class Hierarchy :-

Building (Abstract Base)

ResidentialBuilding (Intermediate)

House (Concrete)

Apartment (Concrete)

TowerBlock (Concrete)

CommercialBuilding (Intermediate)

Shop (Concrete)

Restaurant (Concrete)

Office (Concrete)

Mall (Concrete)

IndustrialBuilding (Intermediate)

Factory (Concrete)

Warehouse (Concrete)
PublicServiceBuilding (Intermediate)
 School (Concrete)
 Hospital (Concrete)
FireStation (Concrete)
PoliceStation (Concrete)

OOP Concepts Summary

Concept	Where Used	Examples
Abstraction	Building base class	Pure virtual functions
Inheritance	All building types	Multi-level inheritance chain
Polymorphism	Virtual functions	Different behavior per type
Encapsulation	Private/protected members	Data hiding, public interface
Composition	Residents in buildings	HAS-A relationships
Association	Employees in buildings	Loose coupling
Method Overriding	All derived classes	Specialized implementations
Static Members	Building counter	Shared across instances

**Virtual
Destructor**

All base classes

Proper cleanup

MODULE 2

CITIZEN & HOUSEHOLD SYSTEM

2.1 Person (Abstract Base Class)

Purpose:

Represents any human in the city. This is the foundation of the person hierarchy. Cannot be instantiated directly.

Attributes :

protected:

string personID; Unique identifier (e.g "PER-00001")

string firstName;
string lastName;
int age; Current age in years
string gender; Male/Female/Other
string dateOfBirth; Format: "YYYY-MM-DD"
string nationalID; Government ID number

Health

int healthStatus; 1-100 scale (100 = perfect health)
bool isAlive; Living status

Location

Building* currentLocation; Where person currently is

Relationships

Person* spouse; Married partner (nullptr if single)
vector<Person*> children; Child pointers
Person* father; Parent pointers
Person* mother;

Private: static int personCounter; For generating unique IDs

Methods :

public:

Constructor

Person(string fname, string lname, int age, string gender);

Pure virtual methods (must be implemented by derived classes)

virtual string getOccupation() const = 0;

virtual double getIncome() const = 0;

virtual void performDailyActivity() = 0;

virtual string getPersonType() const = 0;

Virtual methods (can be overridden)

virtual void age(); Increase age by 1 year

virtual void updateHealth(int change);

virtual void displayInfo() const;

Regular methods (shared by all persons)

string getFullName() const;

int getAge() const;

bool isAdult() const; Age >= 18

bool isChild() const; Age < 18

bool isElderly() const; Age >= 65

Relationships

void marry(Person* partner);

void divorce();

void addChild(Person* child);

bool isMarried() const;

bool hasChildren() const;

Getters/Setters

string getGender() const;

int getHealth() const;

Building* getLocation() const;

void setLocation(Building* location);

Virtual destructor

virtual ~Person();

OOP Concepts Used :

1. Abstraction:

- Person is abstract (has pure virtual functions)
- Cannot create Person objects directly
- Must create Citizen, Student, or Retiree
- Defines interface all person types must implement

2. Encapsulation:

- Private static counter for ID generation
- Protected data accessible to derived classes
- Public interface through methods
- Data validation in setters

3. Self-Referential Class:

- Person contains pointers to other Person objects (family relationships)
- Demonstrates recursive data structures
- Models real-world family trees

4. Association:

- Person associated with Building (current location)
- Weak relationship - person doesn't own building
- Can change location dynamically

2.2 Citizen (Derived from Person)

Purpose:

Represents a working adult citizen. Most common person type in the city.

Attributes :

protected:

string occupation; Job title (e.g "Software Engineer")

string employmentStatus; Employed, Unemployed,

Self-employed

double annualIncome; Yearly salary

double monthlyIncome; Monthly salary

Employment

CommercialBuilding* workplace; Where they work

int yearsExperience; Work experience

string skillLevel; Entry, Mid, Senior, Expert

Financial

double bankBalance; Current savings

double monthlyExpenses; Average spending

double debt; Loans/credit

Residence

ResidentialBuilding* home; Where they live

Household* household; Family unit they belong to

Lifestyle

string educationLevel; High School, Bachelor's, Master's,
PhD
vector<string> skills; Professional skills
int satisfactionLevel; 1-100 (life satisfaction)

private:

double taxesPaid; Annual taxes
vector<string> dailySchedule; Daily routine

Methods :

public:

Constructor

Citizen(string fname, string lname, int age, string gender, string
occupation, double income);

Override pure virtual from Person

string getOccupation() const override;
double getIncome() const override;
void performDailyActivity() override;
string getPersonType() const override;

Override virtual from Person

void age() override; Also handles career progression
void displayInfo() const override;

Employment methods

```
void setJob(string job, CommercialBuilding* workplace, double salary);  
void quitJob();  
void getPromotion(double raise);  
void getFired();  
bool isEmployed() const;
```

Financial methods

```
void earnIncome();           Receive monthly salary  
void payExpenses();          Pay bills  
double payTaxes();           Calculate and pay taxes  
void takeDebt(double amount);  
void payDebt(double amount);  
double getNetWorth() const;   Balance - debt
```

Residence methods

```
void moveToHome(ResidentialBuilding* newHome);  
void leaveHome();  
ResidentialBuilding* getHome() const;
```

Household

```
void joinHousehold(Household* hh);  
void leaveHousehold();  
Household* getHousehold() const;
```

Skills and education

```
void learnSkill(string skill);  
bool hasSkill(string skill) const;  
void increaseEducation(string level);
```

Lifestyle

```
void updateSatisfaction(int change);
```

```
int getSatisfaction() const;
```

```
virtual ~Citizen();
```

OOP Concepts Used :

1. Inheritance:

- Inherits all properties and methods from Person
- Adds citizen-specific attributes
- Implements pure virtual functions from Person

2. Method Overriding:

- Overrides age() to add career progression
- Overrides displayInfo() to show employment details
- Calls parent methods using Person::age()

3. Association (Multiple Types):

- **Aggregation** with CommercialBuilding (workplace) - weak relationship
- **Aggregation** with ResidentialBuilding (home) - weak relationship
- **Association** with Household - can exist independently

4. Real-World Modeling:

- Simulates complete life cycle
- Financial behavior (income, expenses, taxes)
- Career progression over time

2.3 Student (Derived from Citizen)

Purpose:

Represents a student attending school. Specialized type of citizen with limited income and work.

Inheritance:

Person (Grandparent)



Citizen (Parent)



Student (Child)

Attributes :

private:

string studentID;	School-assigned ID
School* school;	School they attend
string gradeLevel; "College"	"Elementary", "Middle", "High", "College"
int currentGrade; college)	Grade number (1-12 or 1-4 for college)
double GPA;	Grade Point Average (0.0-4.0)

struct Course {

string courseName;

string subject; Math, Science, English, etc.

int credits;

double grade; 0-100

string letterGrade; A, B, C, D, F

};

vector<Course> courses;

Academic

int attendancePercentage; 0-100

int studyHoursPerWeek;

bool isFullTime; Full-time or part-time

Part-time work (students can work part-time)

bool hasPartTimeJob;

CommercialBuilding* partTimeWorkplace;

int hoursWorkedPerWeek; Limited hours

double partTimeIncome; Lower than full citizen income

Student life

vector<string> extracurriculars; Clubs, sports, etc.

string major; College major (if applicable)

bool livesOnCampus; For college students

Methods :

public:

 Constructor

 Student(string fname, string lname, int age, string gender,

 School* school, string gradeLevel);

Override from Citizen/Person

string getOccupation() const override; Returns "Student"

double getIncome() const override; Returns part-time
income if any

void performDailyActivity() override; Study, attend class

void displayInfo() const override;

void age() override; Handles grade progression

Academic methods

void enrollInCourse(string name, string subject, int credits);

void dropCourse(string name);

void takeCourse(string courseName, double grade);

void calculateGPA();

bool passedGrade() const;

void advanceGrade();

Work methods

```
void getPartTimeJob(CommercialBuilding* workplace, int  
hours, double pay);
```

```
void quitPartTimeJob();
```

School methods

```
void attendClass();
```

```
void study(int hours);
```

```
void joinClub(string clubName);
```

Graduation

```
bool canGraduate() const;
```

```
void graduate();  Converts to regular Citizen
```

Getters

```
School* getSchool() const;
```

```
double getGPA() const;
```

```
int getGradeLevel() const;
```

```
~Student();
```

OOP Concepts Used :

1. Multi-Level Inheritance:

- Inherits from Citizen, which inherits from Person
- Has access to all ancestor methods and attributes
- Demonstrates inheritance chain

2. Specialized Behavior:

- Student has restricted work (part-time only)
- Different daily activities (school instead of full-time work)
- Shows how derived class modifies base class behavior

3. Lifecycle Transformation:

- Student eventually becomes Citizen (graduation)
- Demonstrates object state transitions

2.4 Retiree (Derived from Citizen)

Purpose:

Represents a retired person no longer working. Receives pension instead of salary.

Inheritance:

Person (Grandparent)



Citizen (Parent)



Retiree (Child)

Attributes :

private:

int retirementAge;	Age when retired
int yearsRetired;	Years since retirement
double pensionAmount;	Monthly pension
double retirementSavings;	Accumulated savings

Former career

string formerOccupation;	Previous job
int yearsWorked;	Total career years

Health (more important for elderly)

bool needsMedicalCare;	
Hospital* preferredHospital;	
double medicalExpenses;	Higher than working citizens

Activities

vector<string> hobbies;	
int socialActivityLevel;	1-100 (how active they are)
bool livesInRetirementHome;	

Family support

bool receivingFamilySupport;

double familySupportAmount; Money from children

Methods :

public:

Constructor

Retiree(string fname, string lname, int age, string gender,
string formerJob, int yearsWorked, double pension);

Constructor from Citizen (for retirement conversion)

Retiree(const Citizen& citizen, double pension);

Override from Citizen/Person

string getOccupation() const override; Returns "Retired"

double getIncome() const override; Returns pension +

family support

void performDailyActivity() override; Leisure activities

void displayInfo() const override;

void age() override; Health deteriorates faster

Retirement methods

void receivePension();

void withdrawSavings(double amount);

void receiveFamilySupport(double amount);

Health

```
void seekMedicalCare();  
void updateHealthStatus();
```

Activities

```
void pursuehobby(string hobby);  
void socialInteraction();
```

Housing

```
void moveToRetirementHome();
```

Legacy

```
bool canAffordLiving() const;
```

```
~Retiree();
```

OOP Concepts Used :

1. Constructor Overloading:

- Multiple constructors for different creation scenarios
- Can create from scratch or convert from Citizen
- Demonstrates constructor flexibility

2. Lifecycle Specialization:

- Different behavior for elderly population
- Higher medical expenses, no work income
- Shows how inheritance handles different life stages

3. Dependency on Other Objects:

- Depends on Hospital for healthcare
- Depends on family (other Person objects) for support
- Demonstrates inter-object relationships

2.5 Household (Composition Class)

Purpose:

Represents a family unit living together. Manages family relationships and shared resources.

Attributes :

private:

string householdID; Unique identifier
vector<Citizen*> members; Family members (aggregation)
ResidentialBuilding* home; Where they live

Financial

double combinedIncome; Total income of all members
double sharedExpenses; Rent, utilities, food, etc.
double savings; Family savings account

Assets

vector<Vehicle*> vehicles; Cars owned by family

vector<string> possessions; Other valuable items

Family structure

Citizen* head; Primary income earner
vector<Student*> dependents; Children/students

Household characteristics

string householdType; "Single", "Nuclear", "Extended",
"Multigenerational"
int householdSize; Number of members
string economicClass; "Low", "Middle", "Upper"

private:

static int householdCounter;

Methods :

public:

Constructor

Household(Citizen* primaryMember, ResidentialBuilding*
residence);

Member management

void addMember(Citizen* member);
void removeMember(Citizen* member);
bool isMember(Citizen* citizen) const;
int getMemberCount() const;
vector<Citizen*> getMembers() const;

Financial management

```
void calculateCombinedIncome();  
void paySharedExpenses();  
void contributeSavings(double amount);  
void withdrawSavings(double amount);  
double getTotalIncome() const;  
double getNetWorth() const;
```

Housing

```
void moveToNewHome(ResidentialBuilding* newHome);  
ResidentialBuilding* getHome() const;  
bool canAffordHome(ResidentialBuilding* home) const;
```

Vehicle ownership

```
void buyVehicle(Vehicle* vehicle);  
void sellVehicle(Vehicle* vehicle);  
int getVehicleCount() const;
```

Family dynamics

```
void setHead(Citizen* head);  
Citizen* getHead() const;  
void addDependent(Student* dependent);
```

Classification

```
void determineHouseholdType();  
void determineEconomicClass();  
string getHouseholdType() const;  
string getEconomicClass() const;
```

Utility

```
void displayHouseholdInfo() const;  
bool isComplete() const;  Has home and at least one member  
  
~Household();
```

OOP Concepts Used :

1. Composition:

- Household is composed of Citizens (HAS-A relationship)
- However, uses **Aggregation** (doesn't own Citizens' lifecycle)
- Citizens can exist without household

2. Association:

- Associated with ResidentialBuilding (home)
- Associated with Vehicle (ownership)
- Manages relationships between multiple objects

3. Static Members:

- Static counter for unique IDs
- Shared across all Household instances

4. Collection Management:

- Uses vectors to manage collections of related objects
- Demonstrates container usage in OOP

5. Business Logic:

- Calculates complex financial relationships
- Models real-world family economics

RELATIONSHIPS BETWEEN CLASSES

1. Person ↔ Person (Self-Referential)

Type: Association (Family relationships)

Nature: Bidirectional

Examples:

- Person has spouse (Person*)
- Person has children (vector<Person*>)
- Person has parents (Person* father, Person* mother)

Implementation:

```
class Person {  
    Person* spouse;  
    vector<Person*> children;  
    Person* father;  
    Person* mother;  
};
```

2. Citizen ↔ CommercialBuilding

Type: Aggregation (Workplace)

Nature: Unidirectional (Citizen knows workplace)

Citizen works at CommercialBuilding

Citizen doesn't own workplace

Workplace can exist without citizen

Implementation:

```
class Citizen {  
    CommercialBuilding* workplace;
```

```
};
```

3. Citizen ↔ ResidentialBuilding

Type: Aggregation (Home)

Nature: Bidirectional

Citizen lives in ResidentialBuilding

ResidentialBuilding contains residents (vector<Citizen*>)

Neither owns the other

Implementation:

```
class Citizen {  
    ResidentialBuilding* home;  
};
```

```
class ResidentialBuilding {  
    vector<Citizen*> residents;  
};
```

4. Student ↔ School

Type: Aggregation

Nature: Bidirectional

Student attends School

School has enrolled students

Neither owns the other

Implementation:

```
class Student {  
    School* school;
```

```
};
```

```
class School {  
    vector<Student*> students;  
};
```

5. Household ↔ Citizen

Type: Aggregation (Strong)

Nature: Bidirectional

Household contains Citizens
Citizen belongs to Household
Citizens can exist independently

Implementation:

```
class Household {  
    vector<Citizen*> members;  
};
```

```
class Citizen {  
    Household* household;  
};
```

6. Household ↔ ResidentialBuilding

Type: Association

Nature: Unidirectional

Household lives in ResidentialBuilding
Building doesn't need to know households

Implementation:

```
class Household {  
    ResidentialBuilding* home;  
};
```

7. Retiree ↔ Hospital

Type: Association

Nature: Unidirectional

Retiree has preferred Hospital

Hospital doesn't track individual retirees

Implementation:

```
class Retiree {  
    Hospital* preferredHospital;  
};
```

MODULE 3

VEHICLE AND TRANSPORTATION

3.1 Vehicle (Abstract Base Class)

Purpose:

Represents any vehicle in the city. Foundation for all vehicle types using abstraction and polymorphism.

Attributes :

protected:

string vehicleID; Unique identifier (e.g., "VEH-00001")

string licensePlate; License plate number

string make; Manufacturer (Toyota, Ford, etc.)

string model; Model name

int year; Manufacturing year

string color; Vehicle color

Location and movement

Building* currentLocation; Where vehicle is now

Building* destination; Where heading to

Road* currentRoad; Road vehicle is on

bool isMoving; Currently in motion

Ownership

Citizen* owner;	Who owns/operates vehicle (nullable)
-----------------	--------------------------------------

Specifications

double fuelCapacity;	Tank size (liters)
----------------------	--------------------

double currentFuel;	Current fuel level
---------------------	--------------------

string fuelType;	Gas, Diesel, Electric
------------------	-----------------------

int maxSpeed;	km/h
---------------	------

int currentSpeed;	Current speed
-------------------	---------------

Condition

int condition;	1-100 scale (100 = perfect)
----------------	-----------------------------

double mileage;	Total kilometers driven
-----------------	-------------------------

int lastServiceMileage;	Mileage at last service
-------------------------	-------------------------

Status

bool isParked;	Currently parked
----------------	------------------

bool isInUse;	Currently being used
---------------	----------------------

private:

static int vehicleCounter; For unique ID generation

Methods :

public:

Constructor

Vehicle(string make, string model, int year, string fuelType,
double fuelCap);

Pure virtual methods (must be implemented by derived classes)

virtual void drive() = 0;

virtual void refuel() = 0;

virtual double getFuelConsumption() const = 0; Liters per 100km

virtual string getVehicleType() const = 0;

virtual int getPassengerCapacity() const = 0;

virtual void displayInfo() const = 0;

Virtual methods (can be overridden)

virtual void startEngine();

virtual void stopEngine();

virtual void park();

virtual void performMaintenance();

Regular methods (shared by all vehicles)

void setDestination(Building* dest);

void updateLocation(Building* location);

```
void increaseMileage(double km);  
bool needsRefueling() const;  
bool needsMaintenance() const;
```

Fuel management

```
void consumeFuel(double amount);  
double getFuelLevel() const;  
double getRemainingRange() const;  How far can travel with  
current fuel
```

Ownership

```
void setOwner(Citizen* newOwner);  
Citizen* getOwner() const;
```

Condition

```
void deteriorate();  Reduce condition over time  
int getCondition() const;
```

Getters

```
string getLicensePlate() const;  
string getMake() const;  
string getModel() const;  
double getMileage() const;  
bool isAvailable() const;  Not in use
```

Virtual destructor

```
virtual ~Vehicle();
```

OOP Concepts Used :

1. Abstraction:

- Vehicle is abstract (has pure virtual functions)
- Cannot instantiate Vehicle directly
- Defines common interface for all vehicle types
- Hides implementation details

2. Encapsulation:

- Private static counter for ID generation
- Protected members accessible to derived classes
- Public interface for external interaction
- Data validation in methods

3. Association:

- Associated with Citizen (owner) - aggregation
- Associated with Building (location)
- Associated with Road (current road)
- Demonstrates multiple relationships

4. State Management:

- Tracks vehicle state (moving, parked, in use)
- Fuel level, condition, location
- Models real-world vehicle behavior

3.2 PrivateCar (Derived from Vehicle)

Purpose:

Personal vehicle owned by citizens for private transportation.

Inheritance:

Vehicle (Base)



PrivateCar (Derived)

Attributes :

private:

int passengerCapacity; Usually 4-7 people
int currentPassengers; Current occupants
vector<Citizen*> passengers; Who's in the car

bool hasAirConditioning;
bool hasGPS;
bool hasSunroof;
string transmissionType; Automatic, Manual

Parking

Building* parkingLocation; Where parked
bool hasAssignedParking; Reserved parking spot

Financial

double purchasePrice; Original cost
double currentValue; Depreciated value
double insuranceCost; Annual insurance
double maintenanceCost; Annual maintenance

Usage

string primaryUse; Commute, Leisure, Business
double averageDailyDistance; km per day

Methods :

public:

 Constructor

 PrivateCar(string make, string model, int year, string fuelType,
 int passengerCap, Citizen* owner);

 Override pure virtual from Vehicle

 void drive() override;

 void refuel() override;

 double getFuelConsumption() const override;

 string getVehicleType() const override;

 int getPassengerCapacity() const override;

 void displayInfo() const override;

 Override virtual from Vehicle

 void startEngine() override;

 void park() override;

 void performMaintenance() override;

 Passenger management

 bool addPassenger(Citizen* passenger);

 void removePassenger(Citizen* passenger);

 void removeAllPassengers();

 int getCurrentPassengerCount() const;

 bool hasSpace() const;

Trip methods

```
void startTrip(Building* destination);
```

```
void endTrip();
```

```
void dropOffPassenger(Citizen* passenger, Building* location);
```

Financial

```
void calculateDepreciation(int years);
```

```
double getAnnualCost() const; // Insurance + maintenance + fuel
```

Parking

```
void parkAt(Building* location);
```

```
Building* getParkingLocation() const;
```

```
~PrivateCar();
```

OOP Concepts Used :

1. Inheritance:

- Inherits all Vehicle properties and methods
- Adds private car-specific features
- Implements all pure virtual functions

2. Method Overriding:

- Provides specific implementation for drive()
- Customizes fuel consumption calculation
- Demonstrates polymorphic behavior

3. Collection Management:

- Manages passengers using vector
- Dynamic passenger addition/removal

4. Financial Modeling:

- Depreciation calculation
- Cost tracking
- Real-world economic behavior

3.3 PublicTransport (Abstract Derived Class)

Purpose:

Base class for public transportation vehicles. Intermediate abstraction level.

Inheritance:

Vehicle (Base)



PublicTransport (Intermediate Abstract)

Attributes :

protected:

int totalCapacity; Maximum passengers

int currentPassengers; Current occupants

vector<Citizen*> passengers; Current riders

double farePrice; Cost per ride
double dailyRevenue; Money collected
int totalRidesGiven; Lifetime rides

Route information

vector<Building*> route; Stops on route
int currentStopIndex; Where on route
string routeName; Route identifier

Schedule

```
struct Schedule {  
    string time; "08:00 AM"  
    Building* stop;  
    bool completed;  
};  
vector<Schedule> schedule;
```

Operational

bool isInService; Currently operating
string operatingHours; "6AM-11PM"
Citizen* driver; Current driver

Methods :

public:

PublicTransport(string make, string model, int year, int capacity,
double fare);

Still pure virtual (must be implemented by concrete types)

virtual void drive() = 0;

virtual void refuel() = 0;

virtual double getFuelConsumption() const = 0;

virtual string getVehicleType() const = 0;

Implemented virtual

int getPassengerCapacity() const override;

void displayInfo() const override;

New virtual methods for public transport

virtual void followRoute() = 0;

virtual void pickupPassengers(Building* stop) = 0;

virtual void dropoffPassengers(Building* stop) = 0;

Route management

void setRoute(const vector<Building*>& stops);

void addStop(Building* stop);

void removeStop(Building* stop);

Building* getNextStop() const;

Passenger management

bool boardPassenger(Citizen* passenger);

void unboardPassenger(Citizen* passenger);

int getAvailableSeats() const;

bool isFull() const;

Revenue

void collectFare(Citizen* passenger);

double getDailyRevenue() const;

Operations

```
void startService();  
void endService();  
void assignDriver(Citizen* driver);  
  
virtual ~PublicTransport();
```

OOP Concepts Used :

1. Intermediate Abstract Class:

- Not fully abstract (implements some methods)
- Not fully concrete (has pure virtual methods)
- Demonstrates layered abstraction

2. Protected Inheritance:

- Children have access to protected route/passenger data
- Encapsulation at multiple levels

3.4 Bus (Derived from PublicTransport)

Purpose:

Public bus following fixed routes.

Inheritance:

Vehicle (Grandparent)



PublicTransport (Parent)

↓
Bus (Child)

Attributes :

private:

string busNumber;	Bus route number (e.g., "42")
bool isDoubleDecker;	Two levels
int standingCapacity;	Additional standing room

bool hasWheelchairAccess;
bool hasBikeRack;
bool hasAirConditioning;

Stops

```
struct BusStop {  
    Building* location;  
    string stopName;  
    int waitingPassengers;  
    double averageWaitTime; Minutes  
};  
vector<BusStop> stops;
```

Statistics

int dailyRiders;	Passengers per day
double onTimePerformance;	% of on-time arrivals
int breakdownCount;	Mechanical failures

Methods :

public:

```
Bus(string make, string model, int year, int capacity,  
    string busNumber, double fare);
```

Override pure virtual

```
void drive() override;  
void refuel() override;  
double getFuelConsumption() const override;  
string getVehicleType() const override;
```

Override PublicTransport virtual

```
void followRoute() override;  
void pickupPassengers(Building* stop) override;  
void dropoffPassengers(Building* stop) override;
```

Bus-specific

```
void openDoors();  
void closeDoors();  
void announceStop(string stopName);  
void updateSchedule();
```

Accessibility

```
void deployWheelchairRamp();  
void loadBicycle();
```

Statistics

```
void recordRider();  
double getAverageRidership() const;  
void updateOnTimePerformance(bool wasOnTime);
```

~Bus();

OOP Concepts Used :

1. Multi-Level Inheritance:

- Inherits from PublicTransport, which inherits from Vehicle
- Access to all ancestor methods and attributes
- Three-level inheritance hierarchy

2. Struct within Class:

- BusStop struct organizes stop information
- Encapsulates related data

3. Simulation Logic:

- Models real bus route behavior
- Passenger pickup/dropoff simulation

3.5 Taxi (Derived from PublicTransport)

Purpose:

On-demand ride service for individual passengers.

Inheritance:

Vehicle (Grandparent)



PublicTransport (Parent)



Taxi (Child)

Attributes :

private:

bool isAvailable;	Free to pick up
bool hasCustomer;	Currently carrying passenger
Citizen* currentCustomer;	Active passenger

Building* pickupLocation;	Where picked up customer
Building* dropoffLocation;	Where dropping off

Pricing

double baseFare;	Initial charge
double perKmRate;	Cost per kilometer
double perMinuteRate;	Cost per minute (in traffic)
double currentFare;	Current trip cost

Statistics

int totalTrips;	Lifetime trips
double totalEarnings;	Lifetime revenue
double averageTripDistance;	Average km per trip
double averageTripDuration;	Average minutes per trip

Rating

double driverRating; 1-5 stars
int ratingCount; Number of ratings

Company
string taxiCompany; Company name (if any)
bool isIndependent; Owner-operator

Methods :

public:

Taxi(string make, string model, int year, Citizen* driver,
double baseFare);

Override pure virtual
void drive() override;
void refuel() override;
double getFuelConsumption() const override;
string getVehicleType() const override;

Override PublicTransport virtual
void followRoute() override; Taxis don't follow fixed routes
void pickupPassengers(Building* stop) override;
void dropoffPassengers(Building* stop) override;

Taxi-specific operations
void acceptRide(Citizen* customer, Building* pickup, Building*
dropoff);
void navigateToPickup();
void pickupCustomer();
void navigateToDropoff();


```
void dropoffCustomer();  
void completeTrip();
```

Pricing

```
void calculateFare(double distance, double duration);  
double getCurrentFare() const;  
void processPayment();
```

Rating

```
void receiveRating(double rating);  
double getDriverRating() const;
```

Availability

```
void goOnline();  
void goOffline();  
bool isAvailableForRide() const;
```

Statistics

```
double getAverageEarnings() const;
```

```
~Taxi();
```

OOP Concepts Used :

1. State Machine:

- Multiple states: available, en route to pickup, carrying passenger
- State transitions based on actions
- Demonstrates state management

2. Dynamic Behavior:

- Taxis don't follow fixed routes (unlike buses)
- Shows polymorphism - same parent, different behavior

3.6 UtilityVehicle (Abstract Derived Class)

Purpose:

Base class for city service vehicles (garbage trucks, ambulances, fire trucks).

Inheritance:

Vehicle (Base)



UtilityVehicle (Intermediate Abstract)

Attributes :

protected:

string department; Fire, Police, Sanitation, Medical

PublicServiceBuilding* homeStation; Base location

bool isOnDuty; Currently working
bool isRespondingToCall; En route to emergency

Crew

vector<Citizen*> crew; Operating personnel
int requiredCrew; Minimum crew size
int maxCrew; Maximum crew size

Service area

int serviceRadius; Coverage area (blocks)
vector<Building*> serviceArea; Buildings covered

Statistics

int totalCallsResponded; Lifetime responses
double averageResponseTime; Average minutes to arrive
int currentCallCount; Calls this shift

Methods :
public:

```
UtilityVehicle(string make, string model, int year,  
               string dept, PublicServiceBuilding* station);
```

Still pure virtual

```
virtual void drive() = 0;  
virtual void refuel() = 0;  
virtual string getVehicleType() const = 0;
```

Implemented virtual

```
double getFuelConsumption() const override;  
int getPassengerCapacity() const override;  
void displayInfo() const override;
```

New pure virtual for utility

```
virtual void respondToCall(Building* location) = 0;  
virtual void performService() = 0;  
virtual void returnToStation() = 0;
```

Crew management

```
void addCrewMember(Citizen* member);  
void removeCrewMember(Citizen* member);  
bool hasFullCrew() const;
```

Duty

```
void startShift();  
void endShift();
```

```
virtual ~UtilityVehicle();
```

OOP Concepts Used :

1. Specialized Abstraction:

- Abstract class for specific domain (utility vehicles)
- Different from general public transport
- Shows multiple levels of abstraction

3.7 GarbageTruck (Derived from UtilityVehicle)

Purpose:

Waste collection vehicle.

Inheritance:

Vehicle (Great-grandparent)



UtilityVehicle (Grandparent)



GarbageTruck (Derived)

Attributes :

private:

double wasteCapacity; Cubic meters

double currentLoad; Current waste loaded

string wasteType; Regular, Recycling, Organic

vector<Building*> collectionRoute; Buildings to collect from

int currentRouteIndex;

bool isCompactorWorking;

double fuelEfficiency; Affected by load

int collectionsToday;

double totalWasteCollected; Lifetime

Methods :

public:

GarbageTruck(string make, string model, int year, double capacity, PublicServiceBuilding* station);

Override pure virtual

void drive() override;

void refuel() override;

string getVehicleType() const override;

Override UtilityVehicle pure virtual

void respondToCall(Building* location) override; Not applicable

void performService() override;

void returnToStation() override;

Collection operations

void followCollectionRoute();

void collectWasteFrom(Building* building);

void compactWaste();

void emptyLoad();

```
bool isFull() const;  
~GarbageTruck();
```

3.8 Ambulance (Derived from UtilityVehicle)

Purpose:

Emergency medical response vehicle.

Inheritance:

Vehicle (Great-grandparent)



UtilityVehicle (Grandparent)



Ambulance (Derived)

Attributes :

private:

bool hasSiren;

bool hasLifeSupport; Advanced medical equipment

Citizen* patient; Current patient

Hospital* destinationHospital;

```
struct MedicalEquipment {
```

```
    string name;
```

```
    bool isOperational;
```

```
    int quantity;  
};  
vector<MedicalEquipment> equipment;
```

Performance

```
int livesTransported;  
int emergenciesResponded;  
double successRate;          % of successful rescues
```

Methods :

public:

```
    Ambulance(string make, string model, int year,  
               PublicServiceBuilding* station);
```

Override pure virtual

```
void drive() override;  
void refuel() override;  
string getVehicleType() const override;
```

Override UtilityVehicle pure virtual

```
void respondToCall(Building* location) override;  
void performService() override;  
void returnToStation() override;
```

Emergency operations

```
void activateSiren();  
void deactivateSiren();
```



```
void loadPatient(Citizen* patient);  
void stabilizePatient();  
void transportToHospital();  
void unloadPatient();
```

```
Equipment  
void checkEquipment();  
void restockSupplies();
```

```
~Ambulance();
```

3.9 FireTruck (Derived from UtilityVehicle)

Purpose:

Fire suppression and rescue vehicle.

Inheritance:

Vehicle (Great-grandparent)



UtilityVehicle (Grandparent)



FireTruck (Derived)

Attributes :

private:

double waterCapacity; Liters
double currentWater; Current water level

bool hasLadder;
int ladderHeight; Meters (if equipped)
bool hasWaterCannon;

int hoseCount;
int axeCount;
bool hasJawsOfLife; Rescue equipment

Statistics
int firesExtinguished;
int peopleRescued;
double averageResponseTime;

Methods :

public:

FireTruck(string make, string model, int year, double waterCap,
PublicServiceBuilding* station);

Override pure virtual
void drive() override;
void refuel() override;
string getVehicleType() const override;

Override UtilityVehicle pure virtual

```
void respondToCall(Building* location) override;  
void performService() override;  
void returnToStation() override;
```

```
    Firefighting operations  
void activateSiren();  
void extendLadder(int height);  
void sprayWater();  
void refillWater();  
void performRescue(Citizen* person);
```

```
    Equipment  
void deployHoses();  
void useAxe();
```

```
~FireTruck();
```

3.10 Road Class (Supporting Infrastructure)

Purpose:

Represents a road connecting buildings. Simple implementation without complex graph structures.

Attributes :

private:

string roadID;

string roadName; "Main Street", "Highway 101"

Connected buildings

Building* startPoint;

Building* endPoint;

Road properties

int lanes; Number of lanes

int speedLimit; km/h

double length; Kilometers

string roadType; "Street", "Avenue", "Highway"

Traffic

int currentVehicles; Vehicles on road now

int capacity; Maximum vehicles

int trafficLevel; 1-10 scale (10 = gridlock)

Condition

int condition; 1-100 (road quality)

bool needsRepair;

private:

static int roadCounter;

Methods :

public:

Road(string name, Building* start, Building* end,
int lanes, int speedLimit);

Vehicle management

void addVehicle(Vehicle* vehicle);
void removeVehicle(Vehicle* vehicle);
bool isFull() const;
int getVehicleCount() const;

Traffic

void updateTrafficLevel();
int getTrafficLevel() const;
bool isCongested() const;

Properties

Building* getStartPoint() const;
Building* getEndPoint() const;
double getLength() const;
int getSpeedLimit() const;

Condition

void deteriorate();

void repair();

bool needsMaintenance() const;

Display

void displayInfo() const;

~Road();

OOP Concepts Used :

1. Association:

- Connects two Building objects
- Bidirectional relationship
- Neither owns the other

2. Simple Container:

- Tracks vehicles on road
- Basic traffic simulation

3.11 Intersection Class

Purpose:

Represents intersection of multiple roads with traffic control.

Attributes :

private:

string intersectionID;

vector<Road*> connectedRoads; Roads meeting here

TrafficLight* signal; Traffic control (composition)

int accidentCount; Historical accidents

bool hasStopSign;

bool hasRoundabout;

Safety

bool hasPedestrianCrossing;

bool hasCrosswalk;

Methods :

public:

Intersection();

void addRoad(Road* road);

void installTrafficLight();

void removeTrafficLight();

void manageTraffic();

bool isSafe() const;

~Intersection();

3.12 TrafficLight Class (State Pattern Implementation)

Purpose:

Traffic signal with different states.

Attributes:

private:

```
string currentState;    "Red", "Yellow", "Green"
int redDuration;        Seconds
int yellowDuration;
int greenDuration;

int timeInCurrentState;
```

Methods :

public:

```
TrafficLight();

void changeToRed();
void changeToYellow();
void changeToGreen();

void update(int seconds);    Advance time
string getCurrentState() const;

~TrafficLight();
```

OOP Concepts Used:

1. State Pattern:

- Different states with different behaviors
- State transitions
- Demonstrates design pattern

OOP CONCEPTS SUMMARY - TRANSPORTATION MODULE

Concept	Where Used	Examples
Multi-Level Inheritance	Vehicle hierarchy	3-4 level deep
Multiple Abstract Levels	PublicTransport, UtilityVehicle	Layered abstraction
Method Overriding	All vehicle types	Specialized behavior
Polymorphism	Virtual functions	Different vehicle behaviors
State Pattern	TrafficLight	State transitions

Association	Vehicle-Citizen, Vehicle-Building	Multiple relationships
Aggregation	Road-Building	Weak ownership
Composition	Intersection-TrafficLight	Strong ownership
Encapsulation	Private members	Data hiding
Struct in Class	Schedule, Equipment	Data organization

MODULE 4

UTILITY SYSTEMS

4.1 UtilityService (Abstract Base Class)

Purpose:

Represents any utility service in the city (power, water, waste).
Foundation for all utility types using abstraction and the **Template Method Pattern**.

Attributes :

protected:

string serviceID;	Unique identifier
string serviceName;	"Power", "Water", "Waste"
string serviceType;	Type of utility

Service coverage

vector<Building*> connectedBuildings;	Buildings receiving service
int totalCustomers;	Number of customers

Production/Capacity

double totalCapacity;	Maximum production/handling
double currentProduction;	Current output
double currentDemand;	Current consumption

Financial

double monthlyRevenue; Income from customers
double operatingCost; Monthly expenses
double maintenanceCost; Equipment maintenance

Service quality

bool isOperational; Currently functioning
double serviceReliability; 0-100% (uptime)
int outageCount; Service interruptions

Infrastructure

vector<Building*> facilities; Plants, stations, centers

Observer pattern - subscribers to notify

vector<Building*> observers;

private:

static int serviceCounter;

Methods :

public:

Constructor

UtilityService(string name, string type, double capacity);

Pure virtual methods (must be implemented)

virtual void generateResource() = 0; Produce utility
virtual void distributeResource() = 0; Send to buildings
virtual double calculateCost() = 0; Usage cost
virtual string getServiceType() const = 0;
virtual void handleOutage() = 0; Emergency response

Virtual methods (can be overridden)

virtual void performMaintenance();
virtual void upgrade(double capacityIncrease);
virtual void displayServiceInfo() const;

Template Method Pattern - common workflow

void processUtilityService(); Template method

Building connection management

void connectBuilding(Building* building);
void disconnectBuilding(Building* building);
bool isConnected(Building* building) const;
int getConnectedBuildingCount() const;

Demand management

void updateDemand();
bool canMeetDemand() const;
double getDemandSatisfaction() const; % of demand met

Observer Pattern methods

void subscribe(Building* building); Building subscribes
void unsubscribe(Building* building); Building unsubscribes
void notifyObservers(string message); Notify all subscribers

Financial

void collectRevenue();
void payOperatingCosts();
double getProfit() const;

Statistics

double getUtilizationRate() const; % capacity used
void recordOutage();

Getters

double getCurrentProduction() const;
double getCurrentDemand() const;
double getCapacity() const;
bool isOperating() const;

Virtual destructor

virtual ~UtilityService();

OOP Concepts Used :

1. **Abstraction:**

- UtilityService is abstract base class
- Defines common utility interface
- Cannot instantiate directly

2. Template Method Pattern:

- processUtilityService() defines algorithm structure
- Derived classes implement specific steps
- Common workflow, different implementations

3. Observer Pattern:

- Buildings observe utility services
- Notified of service changes (outages, rate changes)
- Loose coupling between utilities and buildings

4. Encapsulation:

- Protected data for derived classes
- Private static counter
- Public interface for external access

4.2 PowerService (Derived from UtilityService)

Purpose:

Manages city's electrical power generation and distribution.
Implements **Singleton Pattern** as city has one power grid.

Inheritance:

UtilityService (Base)



PowerService (Derived)

Attributes :

private:

Singleton instance

static PowerService* instance;

static bool instanceExists;

Power generation facilities

vector<PowerPlant*> powerPlants;

vector<PowerStation*> substations;

Grid information

double voltageLevel; Standard voltage (e.g., 220V)

int transmissionLines; Number of power lines

double gridEfficiency; % of power lost in transmission

Energy sources breakdown

double coalPercentage; % from coal

double gasPercentage; % from natural gas

double nuclearPercentage; % from nuclear

double solarPercentage; % from solar

double windPercentage; % from wind

double hydroPercentage; % from hydro

Consumption tracking

double residentialConsumption;

double commercialConsumption;

double industrialConsumption;

Peak/Off-peak

double peakDemand; Maximum demand (usually evening)
double offPeakDemand; Minimum demand (usually night)
string currentPeriod; "Peak" or "Off-Peak"

Pricing (per kWh)
double residentialRate;
double commercialRate;
double industrialRate;

Grid stability
int blackoutCount; Total blackouts
int brownoutCount; Partial outages
double lastOutageDuration; Hours

Renewable integration
bool hasEnergyStorage; Battery systems
double storageCapacity; kWh stored

Private constructor for Singleton
PowerService();

Methods :

public:
 Singleton access method
 static PowerService* getInstance();
 static void deleteInstance();

Delete copy constructor and assignment operator
PowerService(const PowerService&) = delete;
PowerService& operator=(const PowerService&) = delete;

Override pure virtual from UtilityService
void generateResource() override;
void distributeResource() override;
double calculateCost() override;
string getServiceType() const override;
void handleOutage() override;

Override virtual
void performMaintenance() override;
void displayServiceInfo() const override;

Power plant management
void addPowerPlant(PowerPlant* plant);
void removePowerPlant(PowerPlant* plant);
int getPowerPlantCount() const;

Substation management
void addSubstation(PowerStation* station);
vector<PowerStation*> getSubstations() const;

Grid operations
void balanceLoad(); Distribute power evenly
void handlePeakDemand(); Manage high usage periods
void activateBackupPower(); Emergency power

Power generation by source

```
double generateCoalPower();  
double generateGasPower();  
double generateNuclearPower();  
double generateSolarPower();  
double generateWindPower();  
double generateHydroPower();
```

Consumption tracking

```
void trackConsumption(Building* building, double kWh);  
double getTotalConsumption() const;  
double getConsumptionByType(string buildingType) const;
```

Pricing

```
double calculateBill(Building* building, double kWh);  
void adjustRates(double percentageChange);
```

Grid management

```
void triggerBlackout(int durationMinutes);  
void triggerBrownout(int reductionPercent);  
void restorePower();
```

Renewable integration

```
void installEnergyStorage(double capacity);  
void chargeStorage();  
void dischargeStorage(double kWh);
```

Statistics

```
double getRenewablePercentage() const;  
double getCarbonEmissions() const;  
void generatePowerReport() const;
```

~PowerService();

OOP Concepts Used :

1. Singleton Pattern:

- Only one PowerService instance exists
- Global access point via getInstance()
- Private constructor prevents direct instantiation
- Deleted copy constructor/assignment operator

2. Template Method Usage:

- Inherits processUtilityService() from base
- Implements specific steps for power generation

3. Observer Pattern:

- Notifies buildings of power changes
- Buildings react to outages, rate changes

4.3 PowerPlant (Component Class)

Purpose:

Represents individual power generation facility. Used by PowerService.

Attributes :

private:

string plantID;

string plantName;

string energySource; Coal, Gas, Nuclear, Solar, Wind,
Hydro

double capacity; MW (Megawatts)

double currentOutput; Current generation

double efficiency; % efficiency

int age; Years old

int condition; 1-100

double operatingCost; Per day

bool isOperational;

int maintenanceDays; Days of maintenance per year

Methods :

public:

PowerPlant(string name, string source, double capacity);

double generatePower();

void shutdown();

void startup();

void performMaintenance();

double getOperatingCost() const;

double getCapacity() const;

string getEnergySource() const;

~PowerPlant();

4.4 PowerStation (Component Class)

Purpose:

Electrical substation for power distribution.

Attributes :

private:

string stationID;

string location;

double transformerCapacity;

int connectedHomes;

int connectedBusinesses;

bool isOperational;

Methods :

public:

PowerStation(string location, double capacity);

void distributeToArea();

void connectBuilding(Building* building);

~PowerStation();

4.5 WaterService (Derived from UtilityService)

Purpose:

Manages city's water supply and sewage. Implements **Singleton Pattern**.

Inheritance:

UtilityService (Base)



WaterService (Derived)

Attributes :

private:

Singleton instance

static WaterService* instance;

static bool instanceExists;

Water infrastructure

```
vector<WaterSource*> sources;      Rivers, reservoirs, wells  
vector<TreatmentPlant*> treatmentPlants;  
vector<Building*> pumpingStations;
```

Water supply

```
double totalWaterSupply;    Liters per day  
double currentWaterLevel;   % of reservoirs  
double dailyConsumption;    Liters consumed
```

Pipe network

```
int totalPipeLength;        Kilometers  
double pipeCondition;       Average condition 0-100  
int leakCount;              Number of leaks  
double waterLossToLeaks;    % lost to leaks
```

Quality

```
double waterQuality;        0-100 (100 = perfect)  
bool isSafe;                Safe to drink
```

double contaminantLevel; Parts per million

Sewage

double sewageCapacity; Liters per day

double currentSewage; Current sewage volume

bool sewageOverflow; Overflow status

Consumption breakdown

double residentialUsage;

double commercialUsage;

double industrialUsage;

double parkIrrigationUsage;

Pricing (per liter)

double waterRate;

double sewageRate;

Conservation

bool isWaterRationing; // Conservation mode

int rationingLevel; 1-5 (5 = severe)

Weather impact

bool isDrought;

int daysWithoutRain;

Private constructor for Singleton

WaterService();

Methods :

public:

Singleton access

static WaterService* getInstance();

static void deleteInstance();

WaterService(const WaterService&) = delete;

WaterService& operator=(const WaterService&) = delete;

Override pure virtual from UtilityService

void generateResource() override; Collect/treat water

void distributeResource() override; Distribute to buildings

double calculateCost() override;

string getServiceType() const override;

void handleOutage() override; Water main break

Water sources

```
void addWaterSource(WaterSource* source);  
void addTreatmentPlant(TreatmentPlant* plant);
```

Water supply management

```
void collectWater();  
void treatWater();  
double getWaterLevel() const;  
bool hasAdequateSupply() const;
```

Distribution

```
void checkPressure();  
void detectLeaks();  
void repairLeak(int leakID);  
double getWaterPressure() const;
```

Quality control

```
void testWaterQuality();  
void addChemicalTreatment();  
bool isWaterSafe() const;  
void issueBoilWaterAdvisory();
```

Sewage management

```
void collectSewage(Building* building, double liters);  
void treatSewage();  
void handleOverflow();
```

Conservation

```
void implementRationing(int level);
```

```
void liftRationing();  
void encourageConservation();
```

Billing

```
double calculateWaterBill(Building* building, double liters);
```

Weather response

```
void handleDrought();  
void handleFlood();  
void recordRainfall(double mm);
```

Statistics

```
double getConsumptionPerCapita() const;  
void generateWaterReport() const;
```

```
~WaterService();
```

OOP Concepts Used :

1. Singleton Pattern:

- Single water service instance
- Centralized water management
- Same pattern as PowerService

2. Observer Pattern:

- Notifies buildings of water issues

- Boil water advisories
- Rationing announcements

4.6 WaterSource (Component Class)

Purpose:

Represents water supply source (river, reservoir, well).

Attributes :

private:

string sourceID;

string sourceName;

string sourceType; River, Reservoir, Well, Desalination

double capacity; Liters per day

double currentLevel; % full (for reservoirs)

double flowRate; Liters per hour

bool isActive;

double waterQuality; Quality at source

Methods :

public:

 WaterSource(string name, string type, double capacity);

 double collectWater();

 void updateLevel(double change);

 double getCapacity() const;

 ~WaterSource();

4.7 TreatmentPlant (Component Class)

Purpose:

Water treatment facility.

Attributes :

private:

 string plantID;

 string plantName;

double processingCapacity; Liters per day

double currentLoad;

bool isOperational;

double treatmentEfficiency; %

Methods :

public:

 TreatmentPlant(string name, double capacity);

double treatWater(double liters);

void performMaintenance();

bool isOperational() const;

~TreatmentPlant();

4.8 WasteService (Derived from UtilityService)

Purpose:

Manages garbage collection and waste disposal. Implements **Singleton Pattern**.

Inheritance:

UtilityService (Base)



WasteService (Derived)

Attributes :

private:

Singleton instance

static WasteService* instance;

static bool instanceExists;

Waste infrastructure

vector<GarbageTruck*> garbageTrucks;

vector<RecyclingCenter*> recyclingCenters;

Building* landfill;

Building* incinerator;

Waste collection

double dailyWasteGenerated; Tons per day

double dailyWasteCollected; Tons collected

double uncollectedWaste; Backlog

Waste breakdown

double regularWaste; %

double recyclableWaste; %

double organicWaste; %

double hazardousWaste; %

Collection schedule

map<string, int> collectionDays; Area -> day of week

int collectionsPerWeek;

Disposal capacity

double landfillCapacity; Total cubic meters

double landfillUsed; Current usage

double incineratorCapacity; Tons per day

Recycling

double recyclingRate; % of waste recycled

double wasteReduction; % reduction over time

Environmental impact

double methaneEmissions; From landfill

double leachateProduction; Landfill runoff

Pricing

double collectionFee; Per household per month

double overweightFee; Extra charge for excess

Private constructor

WasteService();

Methods :

public:

Singleton access

static WasteService* getInstance();

static void deleteInstance();

WasteService(const WasteService&) = delete;
WasteService& operator=(const WasteService&) = delete;

Override pure virtual from UtilityService

void generateResource() override; Not applicable
void distributeResource() override; Deploy collection trucks
double calculateCost() override;
string getServiceType() const override;
void handleOutage() override; Collection failure

Truck fleet management

void addGarbageTruck(GarbageTruck* truck);
void removeGarbageTruck(GarbageTruck* truck);
int getAvailableTrucks() const;

Collection operations

void scheduleCollection(string area, int dayOfWeek);
void performCollection();
void deployTrucks();

Waste tracking

void trackWasteGeneration(Building* building, double tons);
double getWastePerCapita() const;

Disposal

void disposeAtLandfill(double tons);
void incinerate(double tons);
bool hasDisposalCapacity() const;

Recycling

```
void addRecyclingCenter(RecyclingCenter* center);  
void processRecyclables(double tons);  
void improveRecyclingRate(double increase);
```

Environmental management

```
void manageMethaneEmissions();  
void treatLeachate();  
double getCarbonFootprint() const;
```

Service issues

```
void handleMissedCollection(string area);  
void reportIllegalDumping(Building* location);
```

Billing

```
double calculateWasteBill(Building* building, double tons);
```

Statistics

```
void generateWasteReport() const;  
double getLandfillLifespan() const; // Years remaining
```

```
~WasteService();
```

OOP Concepts Used :

1. Singleton Pattern:

- Single waste service instance

- Centralized waste management
- Consistent with other utilities

2. Observer Pattern:

- Notifies buildings of collection schedule
- Alerts about service disruptions

4.9 RecyclingCenter (Component Class)

Purpose:

Facility for processing recyclable materials.

Attributes :

private:

string centerID;

string centerName;

double processingCapacity; Tons per day

double currentLoad;

Materials accepted

bool acceptsPaper;

bool acceptsPlastic;

bool acceptsGlass;

bool acceptsMetal;

bool acceptsElectronics;

bool isOperational;

double efficiency; % successfully recycled

Methods :

public:

RecyclingCenter(string name, double capacity);

double processRecyclables(double tons);

void sortMaterials();

double getCapacity() const;

~RecyclingCenter();

OOP CONCEPTS SUMMARY - UTILITY MODULE

Concept	Where Used	Examples
Singleton Pattern	All 3 utility services	Single city-wide instance
Template Method Pattern	UtilityService base	Common workflow skeleton
Observer Pattern	Utilities notify buildings	Service alerts, outages
Abstraction	UtilityService base	Pure virtual methods
Encapsulation	Private/protected members	Data hiding
Composition	Services contain components	PowerPlant, WaterSource
Static Members	Singleton instances	Class-level data
Polymorphism	Virtual method overriding	Different utility behaviors
Association	Utilities ↔ Buildings	Service connections

RELATIONSHIPS IN UTILITY MODULE

1. **UtilityService** ↔ **Building** (Association via Observer)

Type: Observer Pattern

Nature: One-to-Many

UtilityService (Subject) has many Building (Observers)

Buildings subscribe to utility services

Notified of service changes

Implementation:

```
class UtilityService {  
    vector<Building*> observers;  
    void notifyObservers(string message);  
};
```

2. **PowerService** ↔ **PowerPlant** (Composition)

Type: Composition (Strong ownership)

Nature: One-to-Many

PowerService owns PowerPlants

PowerPlants cannot exist without PowerService

Implementation:

```
class PowerService {  
    vector<PowerPlant*> powerPlants;  
};
```

3. **WaterService** ↔ **WaterSource** (Composition)

Type: Composition

Nature: One-to-Many

WaterService owns WaterSources

Sources managed by service

Implementation:

```
class WaterService {  
    vector<WaterSource*> sources;  
};
```

4. WasteService ↔ GarbageTruck (Aggregation)

Type: Aggregation

Nature: One-to-Many

WasteService uses GarbageTrucks

Trucks can exist independently (Vehicle system)

Implementation:

```
class WasteService {  
    vector<GarbageTruck*> garbageTrucks;  
};
```

MODULE 5

ECONOMIC & BUDGET SYSTEM

5.1 Business (Abstract Base Class)

Purpose:

Represents any commercial enterprise in the city. Foundation for all business types using abstraction and polymorphism.

Attributes :

protected:

string businessID;	Unique identifier
string businessName;	Company name
string businessType;	Retail, Service, Industrial, Tech
string industry;	Specific industry sector

Location

CommercialBuilding* location; Where business operates

string address;

Ownership

Citizen* owner; Business owner
vector<Citizen*> shareholders; If incorporated

Financial

double initialCapital; Startup investment
double currentCapital; Available funds
double annualRevenue; Yearly income
double annualExpenses; Yearly costs
double annualProfit; Revenue - Expenses

Monthly breakdown

double monthlyRevenue;
double monthlyExpenses;
double monthlyProfit;

Employees

int employeeCount;
vector<Citizen*> employees;
double totalPayroll; Monthly salaries

Business metrics

int yearsInOperation;

string businessStatus; Startup, Growing, Stable, Declining,
Closed

double marketShare; % of market (0-100)

double customerSatisfaction; Rating 0-100

Taxation

double taxRate; Business tax rate

double taxesPaid; Annual taxes

Assets

vector<string> assets; Equipment, vehicles, property

double totalAssetValue;

double liabilities; Debts, loans

Competition

vector<Business*> competitors;

private:

```
static int businessCounter;
```

Methods :

public:

Constructor

```
Business(string name, string type, Citizen* owner,  
        CommercialBuilding* loc, double startupCapital);
```

Pure virtual methods (must be implemented)

```
virtual void operateDaily() = 0;      Daily operations  
virtual double generateRevenue() = 0;  Make money  
virtual double calculateExpenses() = 0; Calculate costs  
virtual string getBusinessCategory() const = 0;  
virtual void displayBusinessInfo() const = 0;
```

Virtual methods (can be overridden)

```
virtual void hireEmployee(Citizen* employee);  
virtual void fireEmployee(Citizen* employee);  
virtual void expandBusiness(double investment);  
virtual void closeBusiness();
```

Financial operations

```
void processMonthlyFinancials();  
void payTaxes();  
double calculateProfit() const;  
double calculateNetWorth() const;  Assets - Liabilities
```

Employee management

```
void payEmployees();  
void promoteEmployee(Citizen* employee, double raise);
```

```
int getEmployeeCount() const;  
double getAverageEmployeeSalary() const;
```

Business operations

```
void investInMarketing(double amount);  
void upgradeEquipment(double cost);  
void takeLoan(double amount, double interestRate);  
void repayLoan(double amount);
```

Competition

```
void addCompetitor(Business* competitor);  
void analyzeMarket();
```

Growth and lifecycle

```
void grow();           Increase size/revenue  
void decline();        Decrease performance  
void updateBusinessStatus();
```

Getters/Setters

```
string getBusinessName() const;  
Citizen* getOwner() const;  
CommercialBuilding* getLocation() const;  
double getRevenue() const;  
double getProfit() const;  
string getBusinessStatus() const;
```

Virtual destructor

```
virtual ~Business();
```

OOP Concepts Used :

1. Abstraction:

- Business is abstract base class
- Cannot instantiate directly
- Defines common business interface
- Forces derived classes to implement specific operations

2. Encapsulation:

- Private business counter
- Protected financial data
- Public interface for operations
- Data validation in methods

3. Association:

- Business → Citizen (owner, employees)
- Business → CommercialBuilding (location)
- Business → Business (competitors)
- Multiple relationship types

5.2 RetailBusiness (Derived from Business)

Purpose:

Store selling physical products to customers.

Inheritance:

Business (Base)



RetailBusiness (Derived)

Attributes :

private:

Inventory management

struct Product {

string productID;

string productName;

string category; Electronics, Clothing, Food, etc.

double costPrice; What business pays

double sellingPrice; What customer pays

int quantity; Stock level

int reorderLevel; When to restock

string supplier;

};

vector<Product> inventory;

Store information

string storeType; Grocery, Electronics, Clothing, etc.

double storeSizeSquareMeters;

Customer data

int dailyCustomers; Average daily foot traffic

double avgTransactionValue; Average purchase amount

int repeatCustomerRate; % returning customers

Sales

double dailySales;
double weeklySales;
double monthlySales;

Inventory costs
double inventoryValue; Total value of stock
double warehouseCost; Storage costs

Marketing
double advertisingBudget;
bool hasLoyaltyProgram;
bool hasOnlineStore;
double onlineSalesPercentage;

Operations
string operatingHours; "9AM-9PM"
int checkoutCounters;
bool hasDeliveryService;

Methods :

public:

 RetailBusiness(string name, Citizen* owner,
CommercialBuilding* loc,
 string storeType, double capital);

Override pure virtual from Business
void operateDaily() override;
double generateRevenue() override;

```
double calculateExpenses() override;  
string getBusinessCategory() const override;  
void displayBusinessInfo() const override;
```

```
Override virtual from Business  
void expandBusiness(double investment) override;
```

```
Inventory management  
void addProduct(string name, string category, double cost,  
                double price, int qty);  
void removeProduct(string productID);  
void updateProductPrice(string productID, double newPrice);  
void restockProduct(string productID, int quantity);  
bool needsRestock(string productID) const;  
void checkInventoryLevels();
```

```
Sales operations  
void processSale(vector<string> productIDs, vector<int>  
quantities);  
double calculateSaleTotal(vector<string> productIDs,  
vector<int> quantities);  
void applyDiscount(string productID, double discountPercent);
```

```
Customer management  
void attractCustomers();  
void implementLoyaltyProgram();  
void trackCustomerSatisfaction();
```

```
Marketing  
void runPromotion(string promotionType, double budget);
```

```
void expandOnline();
```

Analytics

```
Product getBestSellingProduct() const;
```

```
Product getWorstSellingProduct() const;
```

```
double getInventoryTurnover() const; // How fast stock sells
```

```
double getProfitMargin() const;
```

```
~RetailBusiness();
```

OOP Concepts Used :

1. Struct within Class:

- Product struct organizes product data
- Encapsulates related information
- Used in vector for inventory management

2. Complex Business Logic:

- Inventory tracking
- Sales processing
- Profit margin calculations
- Real-world retail operations

5.3 ServiceBusiness (Derived from Business)

Purpose:

Business providing services rather than physical products
(consulting, legal, medical, etc.).

Inheritance:

Business (Base)



ServiceBusiness (Derived)

Attributes :

private:

string serviceType; Consulting, Legal, Medical, Tech, etc.

Service offerings

struct Service {

 string serviceID;

 string serviceName;

 string description;

 double hourlyRate; Price per hour

 int durationHours; Typical duration

 string skillRequired; Required expertise

};

vector<Service> servicesOffered;

Client management

struct Client {

 string clientID;

 Citizen* clientPerson; Or Business*

 vector<string> projectsCompleted;

 double totalBilled;

 bool isActive;

```
};  
vector<Client> clients;
```

Projects

```
struct Project {  
    string projectID;  
    string projectName;  
    Client* client;  
    Service* service;  
    double estimatedCost;  
    double actualCost;  
    int hoursLogged;  
    string status;           Planning, InProgress, Completed  
    string deadline;  
};  
vector<Project> activeProjects;  
vector<Project> completedProjects;
```

Billing

```
double hourlyBillingRate;    Average rate  
int billableHoursPerMonth;  
double outstandingInvoices;  Unpaid bills
```

Reputation

```
double clientSatisfactionScore;  0-100  
int referralCount;               Clients from referrals  
vector<string> testimonials;
```

Methods :

public:

ServiceBusiness(string name, Citizen* owner,
CommercialBuilding* loc,
string serviceType, double capital);

Override pure virtual from Business

void operateDaily() override;
double generateRevenue() override;
double calculateExpenses() override;
string getBusinessCategory() const override;
void displayBusinessInfo() const override;

Service management

void addService(string name, double rate, int duration, string
skill);
void updateServiceRate(string serviceID, double newRate);

Client management

void addClient(Citizen* client);
void removeClient(string clientID);
int getActiveClientCount() const;

Project management

void startProject(string name, Client* client, Service* service);
void workOnProject(string projectID, int hours);
void completeProject(string projectID);
double calculateProjectCost(string projectID) const;

Billing

```
void invoiceClient(Client* client, double amount);  
void receivePayment(Client* client, double amount);  
void sendReminder(Client* client);
```

```
Reputation  
void requestTestimonial(Client* client);  
void improveServiceQuality();
```

```
~ServiceBusiness();
```

5.4 IndustrialBusiness (Derived from Business)

Purpose:

Manufacturing and production business.

Inheritance:

Business (Base)



IndustrialBusiness (Derived)

Attributes :

private:

string productionType; Manufacturing, Processing,
Assembly

Production


```
struct ProductLine {  
    string lineID;  
    string productName;  
    int unitsProducedPerDay;  
    double costPerUnit;  
    double sellingPricePerUnit;  
};  
vector<ProductLine> productionLines;
```

Raw materials

```
struct RawMaterial {  
    string materialID;  
    string materialName;  
    double quantityInStock;    Tons/Units  
    double costPerUnit;  
    double reorderLevel;  
    string supplier;  
};  
vector<RawMaterial> rawMaterials;
```

Production capacity

```
int totalProductionCapacity;    Units per day  
int currentProduction;  
double capacityUtilization;    % capacity used
```

Equipment

```
int machineCount;  
double equipmentValue;  
double maintenanceCost;
```

Quality control

double defectRate; % of defective products
double qualityScore; 0-100

Environmental

double pollutionLevel; Environmental impact
bool hasWasteManagement;
double environmentalFines; Penalties for violations

Methods :

public:

IndustrialBusiness(string name, Citizen* owner,
CommercialBuilding* loc,
 string productionType, double capital);

Override pure virtual from Business

void operateDaily() override;
double generateRevenue() override;
double calculateExpenses() override;
string getBusinessCategory() const override;
void displayBusinessInfo() const override;

Production operations

void manufacture();
void addProductionLine(string productName, int unitsPerDay,
 double cost, double price);
void optimizeProduction();

Materials management

```
void orderRawMaterials(string materialID, double quantity);  
bool hasEnoughMaterials() const;  
void checkMaterialLevels();
```

```
Quality control  
void performQualityCheck();  
void improveQuality(double investment);
```

```
Environmental compliance  
void manageWaste();  
void reduceEmissions(double investment);  
void payEnvironmentalFine(double amount);
```

```
~IndustrialBusiness();
```

5.5 TechStartup (Derived from ServiceBusiness)

Purpose:

Technology company with unique characteristics (rapid growth, venture funding, innovation focus).

Inheritance:

Business (Grandparent)



ServiceBusiness (Parent)



TechStartup (Child)

Attributes :

private:

string productType; SaaS, App, Hardware, AI, etc.

Funding

```
struct FundingRound {  
    string roundName;        Seed, Series A, B, C  
    double amountRaised;  
    vector<string> investors;  
    double valuation;  
    double equityGiven;      % ownership sold  
};
```

```
vector<FundingRound> fundingHistory;
```

double currentValuation;

double burnRate; Monthly cash consumption

int monthsOfRunway; Months until out of money

Product development

string developmentStage; MVP, Beta, Launch, Growth

int activeUsers;

int monthlyActiveUsers;

double userGrowthRate; % monthly growth

Team structure

int engineerCount;

int designerCount;

int salesCount;

int marketingCount;

Metrics (for SaaS)

```
double monthlyRecurringRevenue;  
double customerAcquisitionCost;  
double lifetimeValue;  
double churnRate;           % customers leaving
```

Innovation

```
bool hasPendingPatent;  
int patentsOwned;  
double RnDExpenditure;      Research & Development
```

Methods :

public:

```
TechStartup(string name, Citizen* owner, CommercialBuilding*  
loc,  
             string productType, double capital);
```

Override from Business/ServiceBusiness

```
void operateDaily() override;  
double generateRevenue() override;  
double calculateExpenses() override;  
void displayBusinessInfo() const override;
```

Funding operations

```
void raiseFunding(string roundName, double amount, double  
valuation);  
void calculateRunway();
```

bool needsFunding() const;

Product development

void developProduct();

void launchProduct();

void iterateProduct();

Growth

void acquireUsers(int count);

void improveRetention();

void pivotBusiness(string newProductType);

Innovation

void filePatent(string invention);

void investInRnD(double amount);

Exit strategies

void prepareForIPO();

void seekAcquisition(Business* acquirer, double offerPrice);

~TechStartup();

5.6 CityBudget (Central Financial Management)

Purpose:

Manages the entire city's finances - revenue collection, expense allocation, budget planning. This is a **Singleton** and uses **Facade Pattern** to simplify financial operations.

Attributes :

private:

Singleton

static CityBudget* instance;

static bool instanceExists;

Overall budget

double annualBudget;

double currentBalance;

double totalRevenue;

double totalExpenditure;

double surplus; Positive balance

double deficit; Negative balance

Revenue sources

double propertyTaxRevenue;

double incomeTaxRevenue;

double businessTaxRevenue;

double salesTaxRevenue;

```
double utilityRevenue;    From utility services
double parkingFeesRevenue;
double finesRevenue;      Traffic tickets, violations
double otherRevenue;
```

Expenditure categories

```
double publicSafetyExpense;  Police, Fire
double educationExpense;    Schools
double healthcareExpense;   Hospitals
double infrastructureExpense; Roads, utilities
double administrationExpense; Government operations
double socialServicesExpense; Welfare, assistance
double debtServiceExpense;  Loan repayments
```

Debt management

```
double totalDebt;
double debtLimit;        Maximum allowed debt
vector<Loan> loans;
```

```
struct Loan {
```



```
    string loanID;  
    double principal;  
    double interestRate;  
    int termYears;  
    double monthlyPayment;  
    int paymentsRemaining;  
};
```

Budget allocation per department

```
map<string, double> departmentBudgets;
```

Financial planning

```
int fiscalYear;
```

```
string budgetStatus;      Balanced, Surplus, Deficit
```

Economic indicators

```
double cityGDP;           Total economic output
```

```
double unemploymentRate;
```

```
double averageIncome;
```

```
double povertyRate;
```

Credit rating

string creditRating; AAA, AA, A, BBB, etc.

Private constructor

CityBudget();

Methods :

public:

 Singleton access

static CityBudget* getInstance();

static void deleteInstance();

CityBudget(const CityBudget&) = delete;

CityBudget& operator=(const CityBudget&) = delete;

Revenue collection

void collectPropertyTaxes(double amount);

void collectIncomeTaxes(double amount);

void collectBusinessTaxes(double amount);

void collectSalesTaxes(double amount);

void collectUtilityPayments(double amount);

void collectFines(double amount);

void recordRevenue(string source, double amount);

Expenditure

void allocateBudget(string department, double amount);

```
void spendFromBudget(string department, double amount);  
bool hasAvailableFunds(string department, double amount)  
const;
```

Budget planning

```
void createAnnualBudget();  
void adjustBudget(string department, double newAmount);  
void balanceBudget();
```

Debt operations

```
void issueBond(double amount, double interestRate, int years);  
void takeLoan(double amount, double interestRate, int years);  
void makeDebtPayment();  
double getDebtToGDPRatio() const;  
bool canBorrowMore() const;
```

Financial reporting

```
void generateFinancialReport() const;  
void calculateSurplusDeficit();  
void updateCreditRating();
```

Economic analysis

```
void calculateCityGDP();  
void updateEconomicIndicators();  
double getRevenueGrowth() const;
```

Emergency operations

```
void implementAusterityMeasures();  
void seekEmergencyFunding();
```

Getters

```
double getCurrentBalance() const;  
double getTotalRevenue() const;  
double getTotalExpenditure() const;  
bool isBalanced() const;  
string getCreditRating() const;
```

```
~CityBudget();
```

OOP Concepts Used :

1. Singleton Pattern:

- Single city budget instance
- Centralized financial management
- Global access point

2. Facade Pattern:

- Simplifies complex financial operations
- Single interface for all budget operations
- Hides complexity of revenue/expense tracking

3. STL Map:

- Department budget allocation
- Key-value pairs for organization

5.7 TaxSystem (Helper Class)

Purpose:

Calculates taxes for citizens, businesses, and property. Works with CityBudget.

Attributes :

private:

Tax rates (percentages)

double incomeTaxRateLow; Low income bracket

double incomeTaxRateMid; Middle income

double incomeTaxRateHigh; High income

double propertyTaxRate; % of property value

double businessTaxRate; % of business profit

double salesTaxRate; % of purchases

Tax brackets (income levels)

double lowIncomeCeiling; Max for low bracket

double midIncomeCeiling; Max for mid bracket

Deductions and credits

double standardDeduction;

double childTaxCredit;

double businessExpenseDeduction;

Methods :

public:

TaxSystem();

Tax calculations

double calculateIncomeTax(Citizen* citizen);

double calculatePropertyTax(Building* building);

double calculateBusinessTax(Business* business);

double calculateSalesTax(double purchaseAmount);

Tax policy adjustments

void adjustIncomeTaxRates(double lowRate, double midRate,
double highRate);

void adjustPropertyTaxRate(double newRate);

void adjustBusinessTaxRate(double newRate);

void adjustSalesTaxRate(double newRate);

Tax relief

double applyDeductions(Citizen* citizen, double
taxableIncome);

double applyCredits(Citizen* citizen, double taxOwed);

```
Compliance
void auditBusiness(Business* business);
void penalizeTaxEvasion(Citizen* citizen, double amount);

~TaxSystem();
```

5.8 Transaction (Command Pattern Implementation)

Purpose:

Represents a financial transaction that can be executed, undone, and logged. Implements **Command Pattern**.

Attributes :

private:

```
string transactionID;
string transactionType;    Payment, Transfer, Refund, Tax
```

Parties involved

```
void* payer;              Citizen or Business
void* receiver;           Citizen, Business, or CityBudget
```

double amount;
string description;
string timestamp;

bool isExecuted;
bool isReversed;

Transaction history
string executionTime;
string reversalTime;

Methods :

public:

Transaction(string type, void* from, void* to, double amt, string desc);

Command Pattern methods

void execute(); Perform transaction
void undo(); Reverse transaction
bool canUndo() const;

Information

string getTransactionID() const;
double getAmount() const;


```
string getDescription() const;
bool wasSuccessful() const;

void displayTransactionInfo() const;

~Transaction();
```

OOP Concepts Used :

1. Command Pattern:

- Encapsulates transaction as object
- Supports undo/redo
- Transaction history tracking

2. Polymorphism:

- Payer/receiver can be different types
- Uses void* (simplified - in production use interfaces)

OOP CONCEPTS SUMMARY - ECONOMIC MODULE

Concept	Where Used	Examples
Abstraction	Business base class	Pure virtual methods
Inheritance	Business hierarchy	Multi-level (TechStartup)
Polymorphism	Different business types	Different revenue generation

Encapsulation	Financial data	Private/protected members
Singleton Pattern	CityBudget	Single financial system
Facade Pattern	CityBudget	Simplifies budget operations
Command Pattern	Transaction	Execute/undo operations
Builder Pattern	(Next section)	Complex object construction
Decorator Pattern	(Next section)	Business enhancements
STL Containers	Maps, vectors	Budget allocation, inventory
Struct in Class	Product, Service, Project	Data organization
Association	Business-Citizen-Building	Multiple relationships

FINAL MODULE

EMERGENCY & PUBLIC SERVICE SYSTEM

6.1 EmergencyService (Abstract Base Class)

Purpose:

Represents any emergency response service in the city (Fire, Police, Medical). Foundation for all emergency services using abstraction and the **Chain of Responsibility Pattern**.

Attributes :

protected:

string serviceID;	Unique identifier
string serviceName;	"Fire Department", "Police", "EMS"
string departmentType;	Fire, Police, Medical

Facilities

vector<PublicServiceBuilding*> stations; Fire/Police stations,

Hospitals

int totalStations;

Personnel

int totalStaff;	Total employees
int onDutyStaff;	Currently working
int offDutyStaff;	Not working
vector<Citizen*> staff;	All personnel

struct Shift {

string shiftName;	Morning, Evening, Night
string startTime;	"08:00"

```
    string endTime;          "16:00"  
    vector<Citizen*> assignedStaff;  
    int requiredStaff;  
};  
vector<Shift> shifts;
```

```
Fleet (vehicles)  
vector<UtilityVehicle*> vehicles;  
int totalVehicles;  
int availableVehicles;
```

```
Service area  
double coverageRadius;      Km from each station  
vector<Building*> servicedBuildings;
```

```
Performance metrics  
double averageResponseTime;  Minutes to arrive  
int totalIncidentsResponded;  
int incidentsThisMonth;  
double successRate;          % incidents resolved successfully
```

```
Budget  
double annualBudget;  
double monthlyExpenses;  
bool isFullyFunded;
```

```
Chain of Responsibility  
EmergencyService* nextInChain;  Escalation to higher  
authority
```

private:

static int serviceCounter;

Methods :

public:

Constructor

EmergencyService(string name, string type);

Pure virtual methods (must be implemented)

virtual void respondToIncident(Incident* incident) = 0;

virtual void dispatch(Building* location) = 0;

virtual bool canHandle(Incident* incident) const = 0;

virtual string getServiceType() const = 0;

virtual void performDuties() = 0;

Virtual methods (can be overridden)

virtual void addStation(PublicServiceBuilding* station);

virtual void trainStaff();

virtual void upgradeEquipment(double investment);

virtual void displayServiceInfo() const;

Chain of Responsibility Pattern

void setNextInChain(EmergencyService* next);

void handleIncident(Incident* incident); // Template for chain

Station management

void addStation(PublicServiceBuilding* station);

PublicServiceBuilding* findNearestStation(Building* location)

const;

int getStationCount() const;

Personnel management

void hireStaff(Citizen* person, string role);

void dismissStaff(Citizen* person);

void scheduleShifts();

void assignToShift(Citizen* staff, string shiftName);

int getAvailableStaffCount() const;

Vehicle fleet

void addVehicle(UtilityVehicle* vehicle);

void removeVehicle(UtilityVehicle* vehicle);

UtilityVehicle* getAvailableVehicle() const;

bool hasAvailableVehicle() const;

Response operations

void dispatchUnit(Building* location, UtilityVehicle* vehicle);

void recallUnit(UtilityVehicle* vehicle);

void updateResponseTime(double minutes);

Coverage

bool isInServiceArea(Building* building) const;

void expandCoverage(double additionalRadius);

Budget

void allocateBudget(double amount);

void payMonthlyExpenses();

bool hasAdequateFunding() const;

Statistics

```
void recordIncident();  
void recordSuccess();  
void recordFailure();  
void generatePerformanceReport() const;
```

Getters

```
double getAverageResponseTime() const;  
int getIncidentCount() const;  
double getSuccessRate() const;
```

Virtual destructor

```
virtual ~EmergencyService();
```

OOP Concepts Used :

1. Abstraction:

- EmergencyService is abstract base
- Defines common emergency response interface
- Cannot instantiate directly

2. Chain of Responsibility Pattern:

- Incident passed along chain until handled
- Each service decides if it can handle
- Escalation to higher authority if needed

3. Encapsulation:

- Protected data for derived classes
- Private static counter
- Public interface for operations

4. Template Method:

- handleIncident() defines incident handling workflow
- Derived classes implement specific response

6.2 Incident (Base Class for Events)

Purpose:

Represents an emergency incident requiring response. Base class for different incident types.

Attributes :

protected:

string incidentID; Unique identifier

string incidentType; Fire, Crime, Medical, Accident

Location and time

Building* location; Where incident occurred

string address;

string timestamp; When reported

Severity

int severityLevel; 1-10 (10 = most severe)
bool isCritical; Life-threatening?
string priority; Low, Medium, High, Critical

Status

string status; Reported, Responding, OnScene,
Resolved
bool isActive; Currently ongoing
bool isResolved;

Response

EmergencyService* assignedService;
UtilityVehicle* respondingUnit;
vector<Citizen*> responders; Personnel on scene

Timeline

string reportedTime;
string dispatchedTime;
string arrivedTime;

string resolvedTime;

double responseTime; Minutes from report to arrival

double resolutionTime; Total time to resolve

Casualties and damage

int injuredCount;

int fatalityCount;

double propertyDamage; Dollar value

Reporter

Citizen* reporter; Who called it in

string reportMethod; 911, Witness, Alarm

private:

static int incidentCounter;

Methods :

public:

Constructor

Incident(string type, Building* loc, int severity);

Pure virtual

```
virtual void describe() const = 0;  
virtual double calculateDamage() const = 0;  
virtual string getIncidentCategory() const = 0;
```

Status updates

```
void reportIncident(Citizen* reporter);  
void assignService(EmergencyService* service);  
void dispatchUnit(UtilityVehicle* unit);  
void arriveOnScene();  
void resolveIncident();
```

Timeline tracking

```
void recordDispatchTime();  
void recordArrivalTime();  
void recordResolutionTime();  
void calculateResponseTime();
```

Severity assessment

```
void escalate();           Increase severity  
void deescalate();        Decrease severity  
bool requiresMultipleUnits() const;
```

Casualties

```
void recordInjury(Citizen* person);  
void recordFatality(Citizen* person);  
void recordPropertyDamage(double amount);
```

Information

```
string getStatus() const;
```

```
int getSeverityLevel() const;  
Building* getLocation() const;  
double getResponseTime() const;  
bool isResolved() const;
```

```
Display  
void displayIncidentInfo() const;
```

```
virtual ~Incident();
```

OOP Concepts Used :

1. Base Class for Polymorphism:

- Different incident types inherit from this
- Common interface for all incidents
- Virtual functions for type-specific behavior

2. State Management:

- Tracks incident through lifecycle
- Status transitions (Reported → Responding → Resolved)

6.3 FireIncident (Derived from Incident)

Purpose:

Represents a fire emergency requiring fire department response.

Inheritance:

Incident (Base)



FireIncident (Derived)

Attributes :

private:

string fireType; Residential, Commercial, Industrial,
Wildfire

int fireSize; 1-5 alarm rating

Fire characteristics

string ignitionSource; Electrical, Cooking, Arson, etc.

bool isContained; Spreading or contained

double spreadRate; Sq meters per minute

double burnedArea; Total area burned

Building damage

Building* affectedBuilding;

int floorsAffected;

bool structuralDamage;

bool totalLoss; Building destroyed

Response resources

int firefightersOnScene;

int firetrucksDeployed;

double waterUsed; Liters

Victims

```
int peopleTrapped;  
int peopleRescued;  
int peopleEvacuated;
```

```
    Cause investigation  
bool isArson;  
bool isAccidental;  
string investigationStatus;    Pending, Ongoing, Concluded
```

Methods :

public:

```
    FireIncident(Building* location, string fireType, int alarmLevel);
```

```
    Override pure virtual from Incident  
void describe() const override;  
double calculateDamage() const override;  
string getIncidentCategory() const override;
```

```
    Fire-specific operations  
void assessFireSize();  
void containFire();  
void extinguishFire();  
void rescueVictims();  
void evacuateBuilding();
```

```
    Fire behavior  
void spreadFire();           Fire grows  
void calculateSpread(int minutes);
```

Investigation

```
void investigateCause();  
void determineCause(string cause);
```

Resources

```
void deployFirefighters(int count);  
void deployFiretruck(FireTruck* truck);  
void useWater(double liters);
```

Damage assessment

```
void assessBuildingDamage();  
bool isBuildingHabitable() const;
```

```
~FireIncident();
```

6.4 CrimeIncident (Derived from Incident)

Purpose:

Represents a criminal activity requiring police response.

Inheritance:

Incident (Base)



CrimeIncident (Derived)

Attributes :

private:

string crimeType; Theft, Assault, Burglary, Vandalism,
etc.

string crimeClass; Misdemeanor, Felony

Suspects and victims

```
struct Suspect {  
    string description;  
    bool isIdentified;  
    bool isApprehended;  
    Citizen* person;        If identified  
    string charges;  
};
```

vector<Suspect> suspects;

vector<Citizen*> victims;

vector<Citizen*> witnesses;

Crime details

bool isViolent;

bool isInProgress; Ongoing or completed

bool weaponInvolved;

string weaponType;

Evidence

vector<string> evidence;

bool hasEvidence;

Investigation

string investigationStatus; Open, Active, Cold, Solved, Closed


```
int officersAssigned;  
bool caseSolved;
```

Legal outcome

```
bool arrestMade;  
int arrestCount;  
bool chargesFiled;  
string prosecution;           Pending, Convicted, Acquitted
```

Methods :

public:

```
    CrimeIncident(Building* location, string crimeType, int severity);
```

Override pure virtual from Incident

```
void describe() const override;  
double calculateDamage() const override;  
string getIncidentCategory() const override;
```

Crime-specific operations

```
void addSuspect(string description);  
void addVictim(Citizen* victim);  
void addWitness(Citizen* witness);
```

Investigation

```
void collectEvidence(string evidenceDescription);  
void interviewWitness(Citizen* witness);  
void identifySuspect(Citizen* person);  
void apprehendSuspect(Suspect& suspect);
```

Resolution

```
void makeArrest(Citizen* suspect);  
void fileCharges(string charges);  
void solveCrime();  
void closeCaseUnsolved();
```

Information

```
bool isSolved() const;  
int getSuspectCount() const;  
bool hasArrests() const;
```

```
~CrimeIncident();
```

6.5 MedicalIncident (Derived from Incident)

Purpose:

Represents a medical emergency requiring EMS/hospital response.

Inheritance:

Incident (Base)



MedicalIncident (Derived)

Attributes :

private:

string medicalType;	Heart Attack, Injury, Illness,
Overdose	
string condition;	Critical, Serious, Stable

Patient information

```
struct Patient {  
    Citizen* person;  
    string symptoms;  
    string diagnosis;  
    string treatment;  
    bool requiresHospitalization;  
    bool survived;  
};  
vector<Patient> patients;
```

Medical details

```
bool isLifeThreatening;  
bool requiresSurgery;  
bool requiresICU;
```

Response

```
int paramedicCount;  
Ambulance* respondingAmbulance;  
Hospital* destinationHospital;
```

Treatment

```
string emergencyTreatment;    CPR, First Aid, etc.  
bool patientStabilized;  
string transportMethod;       Ambulance, Helicopter
```

Outcome

```
bool patientSurvived;  
int recoveryTime;             Days
```

Methods :

public:

MedicalIncident(Building* location, string medicalType, int severity);

Override pure virtual from Incident

void describe() const override;

double calculateDamage() const override;

string getIncidentCategory() const override;

Medical operations

void addPatient(Citizen* patient, string symptoms);

void assessPatient(Patient& patient);

void stabilizePatient(Patient& patient);

void administerTreatment(Patient& patient, string treatment);

Transport

void loadIntoAmbulance(Patient& patient);

void transportToHospital(Hospital* hospital);

void admitToHospital(Patient& patient);

Emergency procedures

void performCPR(Patient& patient);

void applyFirstAid(Patient& patient);

void callForHelicopter();

Outcome

void recordRecovery(Patient& patient);

void recordFatality(Patient& patient);

~MedicalIncident();

6.6 FireDepartment (Derived from EmergencyService)

Purpose:

Fire suppression and rescue service.

Inheritance:

EmergencyService (Base)



FireDepartment (Derived)

Attributes :

private:

Specialized resources

int firefighterCount;

int firetruckCount;

int ladderTruckCount;

Equipment

int fireHoseCount;

int respiratorCount;

int hydraulicToolCount; Jaws of life

Training

bool hasHazmatTeam;

bool hasRescueTeam;

bool hasWildlandTeam;

Fire statistics

```
int firesExtinguished;  
int livesRescued;  
int buildingsSaved;  
double totalPropertySaved;  Value saved
```

Response capability

```
int maxSimultaneousIncidents;  How many fires can handle at  
once
```

Methods :

public:

```
FireDepartment(string name);
```

Override pure virtual from EmergencyService

```
void respondToIncident(Incident* incident) override;  
void dispatch(Building* location) override;  
bool canHandle(Incident* incident) const override;  
string getServiceType() const override;  
void performDuties() override;
```

Fire-specific operations

```
void fightFire(FireIncident* fire);  
void rescueVictims(FireIncident* fire);  
void containBlaze(FireIncident* fire);  
void extinguishFire(FireIncident* fire);
```

Training

```
void trainFirefighters();  
void conductFireDrill();  
void certifyHazmat();
```

Prevention

```
void inspectBuilding(Building* building);  
void educatePublic();
```

```
~FireDepartment();
```

6.7 PoliceDepartment (Derived from EmergencyService)

Purpose:

Law enforcement and crime prevention.

Inheritance:

EmergencyService (Base)



PoliceDepartment (Derived)

Attributes :

private:

Personnel

int officerCount;

int detectiveCount;

int swatCount;

Vehicles

```
int patrolCarCount;  
int motorcycleCount;
```

Equipment

```
bool hasK9Unit;  
bool hasSWATTeam;  
bool hasForensicsLab;
```

Crime statistics

```
int crimesInvestigated;  
int arrestsMade;  
int casesSolved;  
double clearanceRate;      % of crimes solved
```

// Patrol

```
map<string, int> patrolZones;  Zone → officers assigned
```

Detention

```
int detentionCells;  
int currentDetainees;
```

Methods :

```
public:
```

```
    PoliceDepartment(string name);
```

Override pure virtual from EmergencyService


```
void respondToIncident(Incident* incident) override;  
void dispatch(Building* location) override;  
bool canHandle(Incident* incident) const override;  
string getServiceType() const override;  
void performDuties() override;
```

Crime response

```
void investigateCrime(CrimeIncident* crime);  
void pursueSuspect(CrimeIncident* crime);  
void makeArrest(CrimeIncident* crime, Citizen* suspect);
```

Patrol

```
void deployPatrol(string zone);  
void conductPatrol();
```

Investigation

```
void collectEvidence(CrimeIncident* crime);  
void interrogateSuspect(Citizen* suspect);  
void closeCase(CrimeIncident* crime);
```

Prevention

```
void increasePatrols(string zone);  
void communityPolicing();
```

```
~PoliceDepartment();
```

6.8 MedicalService (Derived from EmergencyService)

Purpose:

Emergency medical services and hospital coordination.

Inheritance:

EmergencyService (Base)



MedicalService (Derived)

Attributes :

private:

Personnel

int paramedicCount;

int EMTCount;

int doctorCount;

Vehicles

int ambulanceCount;

int helicopterCount;

Hospitals

vector<Hospital*> hospitals;

Medical statistics

int emergenciesResponded;

int patientsTransported;

int livesSaved;

double survivalRate; % patients survived

Equipment

```
bool hasAirAmbulance;  
bool hasTraumaCenter;
```

Methods :

public:

```
MedicalService(string name);
```

```
Override pure virtual from EmergencyService  
void respondToIncident(Incident* incident) override;  
void dispatch(Building* location) override;  
bool canHandle(Incident* incident) const override;  
string getServiceType() const override;  
void performDuties() override;
```

```
Medical response  
void treatPatient(MedicalIncident* medical);  
void stabilizeOnScene(MedicalIncident* medical);  
void transportToHospital(MedicalIncident* medical);
```

```
Hospital coordination  
Hospital* findNearestHospital(Building* location) const;  
bool hasAvailableBed(Hospital* hospital) const;
```

```
~MedicalService();
```

6.9 DispatchSystem (Coordinator Class)

Purpose:

Central coordination of emergency responses. Routes incidents to appropriate services. Implements **Mediator Pattern**.

Attributes :

private:

Services

FireDepartment* fireDept;

PoliceDepartment* policeDept;

MedicalService* medicalService;

Active incidents

vector<Incident*> activeIncidents;

vector<Incident*> resolvedIncidents;

Call center

int operatorCount;

int incomingCalls;

```
queue<Incident*> incidentQueue;
```

Statistics

```
int totalCalls;
```

```
int callsThisMonth;
```

```
double avgDispatchTime;    Seconds to dispatch
```

Methods :

public:

```
DispatchSystem();
```

Service registration

```
void registerFireDepartment(FireDepartment* dept);
```

```
void registerPoliceDepartment(PoliceDepartment* dept);
```

```
void registerMedicalService(MedicalService* service);
```

Incident handling (Mediator Pattern)

```
void receive911Call(Incident* incident);
```

```
void routeIncident(Incident* incident);
```

```
void dispatchAppropriateService(Incident* incident);
```

Multi-service incidents

```
void dispatchMultipleServices(Incident* incident);
```

Queue management

```
void processQueue();
```

```
void prioritizeIncidents();
```

Coordination

```
void coordinateResponse(Incident* incident);
```

```
void updateStatus(Incident* incident, string newStatus);
```

Statistics

```
void recordCall();
```

```
void generateDispatchReport() const;
```

```
~DispatchSystem();
```

OOP Concepts Used :

1. Mediator Pattern:

- DispatchSystem coordinates between services
- Services don't communicate directly
- Central point of control
- Reduces coupling between services

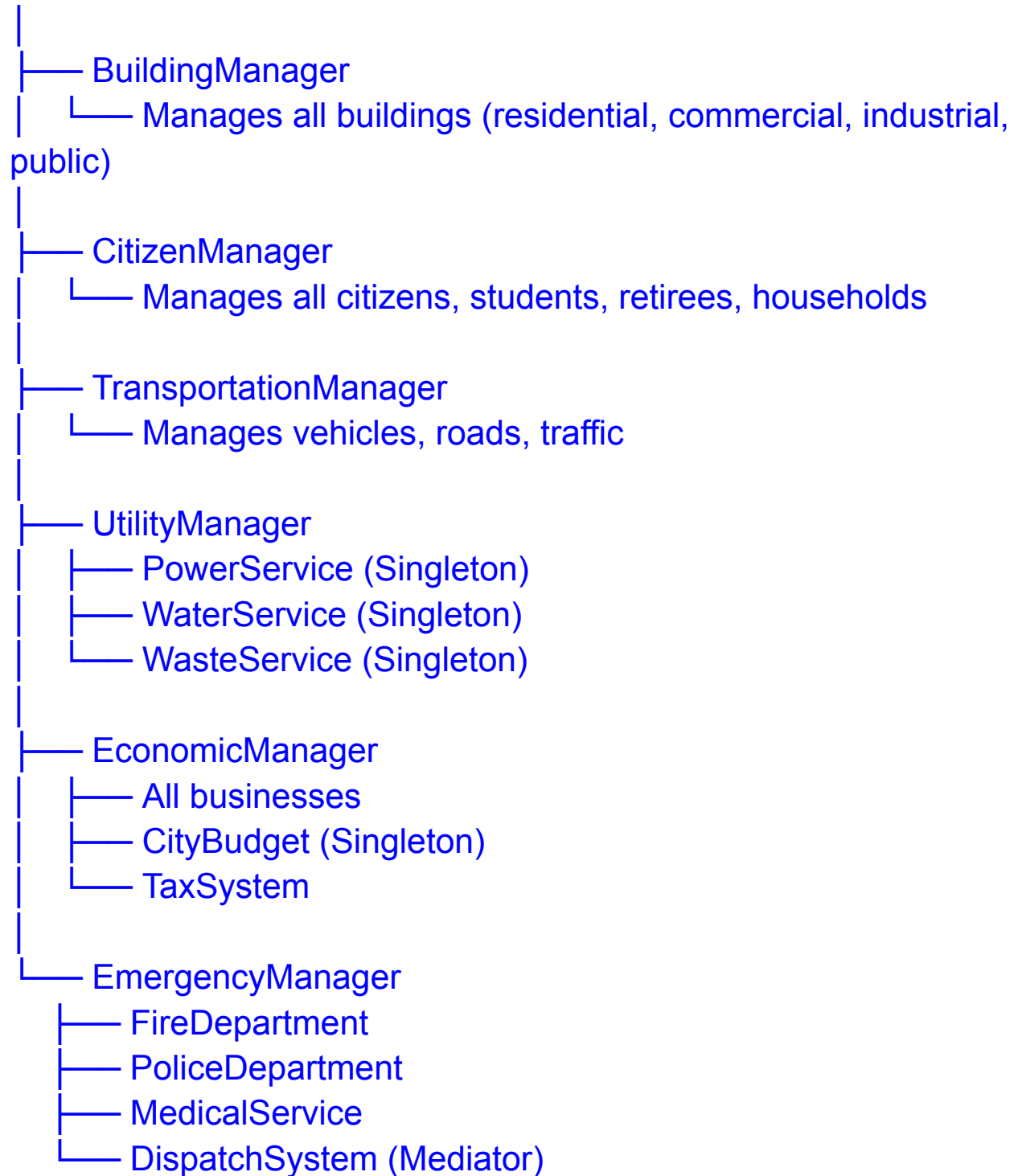
OOP CONCEPTS SUMMARY - EMERGENCY MODULE

Concept	Where Used	Examples
Abstraction	EmergencyService, Incident	Pure virtual methods

Inheritance	Service & Incident hierarchies	Multi-level inheritance
Polymorphism	Different services, incidents	Type-specific behavior
Chain of Responsibility	EmergencyService	Incident escalation
Mediator Pattern	DispatchSystem	Service coordination
Template Method	handleIncident()	Common workflow
Dynamic Casting	Service type checking	Runtime type identification
Encapsulation	Protected/private members	Data hiding
Association	Service ↔ Incident	Response relationships
Composition	Incident contains Patients	Strong ownership
State Management	Incident status tracking	Lifecycle modeling
STL Containers	Vectors, queues, maps	Data organization

How All 6 Modules Connect:

CityManager (Central Controller - Singleton, Facade)



CONCLUSION :

These are all my modules in my OOP Project that will a Smart City Automation Tool and describes how a city runs in daily routine and what are the necessary things that should be present in a city .
